# Client Interface (CIF) Full Microarchitecture Specification

## Contents

# 1 Overview

The Controller Interface (CIF) is the AXI-facing front-end of the memory subsystem. It translates AXI4 read and write transactions into normalized, fixed-width packets suitable for MC Core processing. The CIF handles burst splitting, transaction tagging, packet formation, response tracking, and in-order AXI response generation. MC Core remains transaction-agnostic and operates purely on a per-packet basis.

## 1.1 Global Assumptions

- AXI4 protocol.
- Data width: 512 bits (64 bytes). Parameterizable via RTL.
- Address width: 32 bits (FPGA initial configuration). Parameterizable.
- AXID width: 6 bits (64 unique AXI IDs).
- Supported burst types: INCR and WRAP. FIXED not supported.
- Multiple outstanding transactions supported per AXID (up to 4).
- Maximum burst length: 256 beats (AXI4 maximum).
- Narrow INCR and unaligned INCR transfers supported.
- Narrow WRAP and unaligned WRAP transfers not supported.
- Out-of-order (OoO) completions from MC Core are supported and resolved within CIF.
- Protocol violations (illegal burst type, unaligned WRAP, etc.) are design-time assertions only. Runtime validation is limited to address legality checks.

# 2 Parameters

| Parameter | Default | Description |
|---|---|---|
| AXID_W | 6 | AXI ID width. Supports up to $2^6 = 64$ unique IDs. |
| ADDR_W | 32 | Address width in bits. |
| DATA_W | 512 | Data path width in bits. |
| N_AR | 128 | AR Metadata FIFO depth ($= 4 \times$ N_CLIENTS). |
| N_AW | 128 | AW Metadata FIFO depth ($= 4 \times$ N_CLIENTS). |
| N_W | 64 | W Data FIFO depth. |
| N_PKT | 64 | Packet FIFO depth (read and write). |
| MC_RD_BUF | 64 | MC Core internal read buffer depth. Defines ROB depth. |
| ROB_WATERMARK | 26 | MC-side watermark for backpressure signalling. |
| MAX_OUTSTANDING | 4 | Maximum outstanding transactions per AXID. |
| N_CLIENTS | 32 | AXI client count. Fixed at compile time (max 32). |
| All parameters are configurable at synthesis time. | | |

Table 1: CIF Global Parameters

**Note on N_CLIENTS.** The number of AXI clients is fixed at synthesis time. Each client is statically allocated MAX_OUTSTANDING = 4 slots in the AR and AW metadata FIFOs, giving N_AR = N_AW = 4 $\times$ N_CLIENTS. Idle clients retain their allocated slots; dynamic reallocation across clients is deferred to a future CIF revision.

# 3 Write Path

## 3.1 Overview

The write path accepts AXI write transactions on the AW and W channels, validates them, splits bursts into normalized 64-byte packets, and forwards them to MC Core. Per-packet completions are tracked and aggregated to generate a single AXI B channel response per transaction.

## 3.2 Block Descriptions

**AW Metadata FIFO (depth = N_AW = 4× N_CLIENTS).** Captures and buffers incoming AW channel metadata: `AWADDR`, `AWLEN`, `AWSIZE`, `AWBURST`, `AWID`. Decouples AXI request arrival from downstream validation and scheduling. Each AXI client is statically allocated 4 slots. `AWREADY` is driven low when no slots are available for the requesting client.

**W Data FIFO (depth = N_W).** Captures and buffers AXI write data beats: `WDATA`, `WSTRB`, `WLAST`. Operates independently of the AW channel. W beats flow in freely regardless of AW validation status.

**Request Validator.** Pops entries from the AW Metadata FIFO and performs address legality checks combinationally:

- Address range validation.

Protocol-level constraints (burst type, WRAP legality, alignment, burst length) are enforced as design-time assertions and are not checked at runtime — compliant AXI masters are assumed. Valid requests are forwarded to the Transaction Tag Allocator. Address errors are forwarded to the Error Handler along with `AWID`.

**Transaction Tag Allocator.** Shared across all `N_CLIENTS` AXI clients. Allocates a unique `txn_tag` per validated transaction:

```
txn_tag[7:0] = AXID[5:0] + seqnum[1:0]
seqnum: monotonically increasing per AXID, wraps at 4
```

Accepts inputs from both the Request Validator (valid requests) and the Error Handler (NOP transactions). Seqnum allocation is blocked when all 4 outstanding slots for an AXID are occupied. Tags are freed when the Response Tracker emits a BRESP for the corresponding transaction.

**Burst Splitter (two-stage pipeline).** Shared across all `N_CLIENTS` AXI clients. Converts validated AXI burst requests into normalized beat-level packets.

- *Stage 1 — Row Boundary Split:* Splits bursts such that no sub-transaction crosses a DRAM row boundary. Generates row-confined sub-transactions.
- *Stage 2 — Command Alignment & Packetization:* Aligns packets to memory-command boundaries. Generates byte-enable wrapper masks for partial or edge-aligned packets.

Internal state: `base_addr`, `beat_idx`, `beats_left`, `burst_type`, `burst_len`, `txn_tag`.

**Packet Formation Unit.** Assembles normalized 618-bit write packets:

```
pkt_type    [1:0]     2 bits  (00=Write, 01=Read, 10=Err_write, 11=Err_read)
txn_tag     [9:2]     8 bits
addr        [41:10]  32 bits
```

```
data        [553:42] 512 bits
wrapper     [617:554]  64 bits  (1 bit = 1 byte validity)
Total: 618 bits
```

One AXI beat maps directly to one normalized packet. No beat accumulation required.

**Packet FIFO (depth = N_PKT).** Buffers normalized write packets before transfer to MC Core. Provides decoupling between CIF packet generation rate and MC Core processing rate.

**Response Tracker.** Shared across all `N_CLIENTS` AXI clients. Tracks outstanding write transactions on a per-tag basis. Stores: `txn_tag`, `AXID` (recovered as `txn_tag[7:2]`), `expected_packets` (from `AWLEN+1`), `completed_packets`, `status`. Emits a single `BRESP` when:

```
completed_packets == expected_packets
```

On receiving an error status from MC Core (corresponding to an `Err_write` packet), passes `DECERR` to the B Response Generator.

**B Response Generator.** Drives the AXI B channel: `BVALID`, `BRESP`, `BID`. `BID` is recovered from `txn_tag[7:2]`. Generates `DECERR` when an error status is received from the Response Tracker, and `OKAY` otherwise. Single path to the AXI B channel — no arbitration mux required.

**Error Handler.** Receives address error notifications from the Request Validator. Sets `pkt_type = Err_write` and forwards the transaction into the normal pipeline via the Transaction Tag Allocator. W beats for the transaction flow through the pipeline without draining or discarding. MC Core receives the `Err_write` packet and returns an error completion; `DECERR` is subsequently generated by the B Response Generator on the AXI B channel.

### 3.3 Write Packet Format

| Field | Width (bits) | Description |
|---|---|---|
| `pkt_type` | 2 | Packet type: `00=Write, 01=Read, 10=Err_write, 11=Err_read` |
| `txn_tag` | 8 | Transaction tag (`AXID[5:0] + seqnum[1:0]`) |
| `addr` | 32 | Write address |
| `data` | 512 | Write data payload |
| `wrapper` | 64 | Byte-level validity mask (1 bit = 1 byte) |
| **Total** | **618** | `PKT_DATA[617:0]` |

Table 2: Write Packet Format

### 3.4 Write Path Backpressure Chain

Backpressure propagates right-to-left through the write pipeline:

```
MC Core busy -> PKT_READY = 0
    -> Packet FIFO fills up
    -> Packet Formation Unit stalls
    -> Burst Splitter stalls
    -> W Data FIFO absorbs backpressure
    -> W Data FIFO full -> WREADY = 0
```

—> AW Metadata FIFO full —> AWREADY = 0

This avoids materializing large bursts upfront and keeps buffering requirements bounded.

# 4 Read Path

## 4.1 Overview

The read path accepts AXI read transactions on the AR channel, validates them, splits bursts into normalized read request packets, forwards them to MC Core, and reassembles in-order read data for AXI R channel delivery. MC Core may return completions out of order; the CIF Reorder Buffer restores per-AXID AXI ordering before response generation.

## 4.2 Block Descriptions

**AR Metadata FIFO (depth = N_AR = 4× N_CLIENTS).** Captures and buffers incoming AR channel metadata: `ARADDR`, `ARLEN`, `ARSIZE`, `ARBURST`, `ARID`. Decouples AXI request arrival from downstream processing. Each AXI client is statically allocated 4 slots. `ARREADY` is driven low when no slots are available for the requesting client.

**Request Validator.** Performs address legality checks only. Protocol-level constraints are enforced as design-time assertions and are not checked at runtime. Valid requests proceed to the Transaction Tag Allocator. Address errors are forwarded to the Error Handler with `ARID` preserved.

**Transaction Tag Allocator.** Shared across all `N_CLIENTS` AXI clients. Same encoding as the write path:

```
txn_tag[7:0] = AXID[5:0] + seqnum[1:0]
```

Accepts inputs from both the Request Validator (valid requests) and the Error Handler (NOP transactions). Seqnum is monotonically increasing per AXID and wraps at 4.

**Burst Splitter.** Shared across all `N_CLIENTS` AXI clients. Same two-stage pipeline as write path. Generates per-beat read request contexts containing `txn_tag` and `addr`.

**Packet Formation Unit.** Assembles normalized 42-bit read request packets:

```
pkt_type    [1:0]     2 bits   (00=Write, 01=Read, 10=Err_write, 11=Err_read)
txn_tag     [9:2]     8 bits
addr        [41:10]  32 bits
Total: 42 bits        PKT_DATA[41:0]
```

**Packet FIFO (depth = N_PKT).** Buffers normalized read request packets before forwarding to MC Core.

**Read Transaction Table (RTT).** Allocated at tag assignment time. Stores per-transaction metadata:

- `AXID`
- `expected_packets` (from `ARLEN+1`)
- `completed_packets`

Asserts `rob_last` when `completed_packets == expected_packets`, signalling the final beat of a transaction to Merge Logic. Entry is freed after `rob_last` is emitted.

**Reorder Buffer (ROB).** Stores out-of-order read completions from MC Core and emits them in-order per AXID. Depth is tied to `MC_RD_BUF` — the ROB never needs more slots than MC Core can have in-flight simultaneously.

Structure:

```
ROB[MC_RD_BUF]  -- flat buffer, depth = 64
Each slot:
    valid      1 bit
    data     512 bits
    status     4 bits

Free list: 64 entries
Lookup table: (AXID, seqnum) -> slot index
```

On tag allocation:

```
pop free slot index from free list
store mapping: (AXID, seqnum) -> slot index in lookup table
```

On MC completion:

```
extract AXID = tag[7:2], seqnum = tag[1:0]
lookup slot index from (AXID, seqnum)
ROB[slot].data   = completion data
ROB[slot].status = completion status
ROB[slot].valid  = 1

emit loop:
    while ROB[slot_at_head].valid == 1:
        emit to Merge Logic
        ROB[slot_at_head].valid = 0
        push slot back to free list
        advance head pointer for AXID
```

Depth: `MC_RD_BUF = 64`. Implemented as dual-port SRAM/BRAM. Watermark threshold: `MC_RD_BUF - ROB_WATERMARK = 64 - 26 = 38`.

**Merge Logic.** Receives in-order beats from the ROB and performs per-ID beat sequencing. Streams ordered `RDATA` to the R Response Generator.

**R Response Generator.** AXI R channel driver. Restores `RID` from `txn_tag[7:2]`, generates `RLAST`, `RRESP`, asserts `RVALID`, and drives `RDATA`. Generates `DECERR` when an error status is received from the ROB, and `OKAY` otherwise. Single path to the AXI R channel — no arbitration mux required.

**Error Handler.** Receives address error notifications from the Request Validator with `ARID` preserved. Sets `pkt_type = Err_read` and forwards the transaction into the normal pipeline via the Transaction Tag Allocator. MC Core receives the `Err_read` packet and returns an error completion; `DECERR` is subsequently generated by the R Response Generator on the AXI R channel.

### 4.3 Read Packet Format

| Field | Width (bits) | Description |
|---|---|---|
| pkt_type | 2 | Packet type: 00=Write, 01=Read, 10=Err_write, 11=Err_read |
| txn_tag | 8 | Transaction tag (AXID[5:0] + seqnum[1:0]) |
| addr | 32 | Read address |
| **Total** | **42** | PKT_DATA[41:0] |

Table 3: Read Packet Format

### 4.4 Read Path Backpressure Chain

MC Core does not support backpressure on the completion interface (RESP_READY must always be 1). CIF must therefore ensure the ROB never fills completely. Backpressure is applied by halting new read requests to MC Core when the ROB occupancy reaches the watermark threshold:

```
ROB occupancy >= MC_RD_BUF - ROB_WATERMARK  (= 64 - 26 = 38)
    -> PKT_READY = 0  (stop sending new requests to MC)
    -> Packet FIFO fills up
    -> Burst Splitter stalls
    -> AR Metadata FIFO absorbs backpressure
    -> AR Metadata FIFO full -> ARREADY = 0
```

The ROB_WATERMARK = 26 headroom ensures all in-flight requests already accepted by MC Core can return completions without overflowing the ROB.

## 5 CIF ↔ MC Core Interface

### 5.1 Write Interface (CIF → MC Core)

| Signal | Width | Description |
|---|---|---|
| PKT_VALID | 1 | Packet valid |
| PKT_READY | 1 | MC Core ready to accept |
| PKT_DATA | 618 | Serialized write packet |

Table 4: Write Request Interface

### 5.2 Read Interface (CIF → MC Core)

| Signal | Width | Description |
|---|---|---|
| PKT_VALID | 1 | Packet valid |
| PKT_READY | 1 | MC Core ready to accept |
| PKT_DATA | 42 | Serialized read request packet |

Table 5: Read Request Interface

### 5.3 Completion Interface (MC Core → CIF)

Agreed with MC Core team. `RESP_READY` must be held high at all times by CIF — MC Core does not support backpressure on the completion path.

| Signal | Width | Description |
|---|---|---|
| `RESP_VALID` | 1 | Completion valid |
| `RESP_READY` | 1 | Must always be 1 (CIF always ready) |
| `resp_type` | 2 | `00=RD_DATA, 01=WR_ACK, 10=ERR` |
| `tag` | 8 | `tag[7:2]=AXID, tag[1:0]=seqnum` |
| `data` | 512 | Read data (valid for `RD_DATA` only) |
| `status` | 4 | Status/error code (encoding TBD with MC team) |

Table 6: MC Core Completion Interface — agreed with MC Core team

### 5.4 Packet Type Encoding

The `pkt_type` field is carried in bits `[1:0]` of all CIF-to-MC-Core packets. Agreed with MC Core team.

```
pkt_type[1:0]:
    00 = Write     — normal write transaction
    01 = Read      — normal read transaction
    10 = Err_write — address-error write; MC Core returns error completion
    11 = Err_read  — address-error read;  MC Core returns error completion
```

MC Core discards data for `Err_write` and `Err_read` packets and returns an error status on the completion interface. CIF maps this to `DECERR` on the AXI B or R channel respectively.

### 5.5 Tag Encoding

```
txn_tag[7:0]:
    [7:2] = AXID   (6 bits, up to 64 unique IDs)
    [1:0] = seqnum (2 bits, 0-3, per-AXID outstanding slot)
```

Agreed with MC Core team. MC Core returns the same `tag` value on completion that CIF embedded in the original request packet.

## 6 Edge Cases and Design Decisions

### 6.1 Invalid Request Handling — Write Path

The Request Validator performs address legality checks only at runtime. On an address error, the Error Handler sets `pkt_type = Err_write` and injects the transaction into the normal pipeline via the Transaction Tag Allocator. W beats for the transaction flow through the W Data FIFO and Packet Formation Unit without modification. MC Core receives the `Err_write` packet, discards the data, and returns an error completion. The B Response Generator drives `BRESP = DECERR` on the AXI B channel. No W beat draining or direct error bypass path is required.

### 6.2 Invalid Request Handling — Read Path

On an address error, the Error Handler sets `pkt_type = Err_read` and injects the transaction into the normal pipeline. MC Core receives the `Err_read` packet and returns an error completion. The R Response Generator drives `RRESP = DECERR` on the AXI R channel. No ROB slot is bypassed and no special read-path handling is required.

### 6.3 Protocol Violation Handling

Burst type violations, WRAP illegality, alignment errors, and maximum burst length violations are not checked at runtime. Compliant AXI4 masters are assumed. These constraints are enforced as design-time assertions in the verification environment only.

### 6.4 Error Response Path

Both read and write paths use a single response path — the normal Response Generator drives the AXI R and B channels respectively. The arbitration mux previously used to multiplex error and normal responses has been removed. All responses, including `DECERR`, are generated by the Response Generator based on the completion status received from MC Core.

### 6.5 Out-of-Order Completion Handling

MC Core may return read completions out of order. The ROB absorbs OoO completions using a flat buffer indexed by dynamically allocated slot. On completion, CIF resolves the slot index via a lookup table keyed on `(AXID, seqnum)`. In-order emission per AXID is maintained by tracking a head pointer per AXID; a slot is emitted and freed only when the head slot for that AXID is valid. This guarantees AXI R channel ordering compliance.

### 6.6 ROB Sizing and Overflow Prevention

The ROB depth is tied to `MC_RD_BUF` — the MC Core internal read buffer depth. The ROB never needs more entries than MC Core can have in-flight simultaneously. With `MC_RD_BUF = 64`, the ROB is 64 entries deep. Since `RESP_READY` must always be 1, CIF cannot stall incoming completions. Instead, CIF halts new read request issuance (`PKT_READY = 0`) when ROB occupancy reaches `MC_RD_BUF - ROB_WATERMARK = 38`. The `ROB_WATERMARK = 26` headroom absorbs all completions in-flight within MC Core without ROB overflow.

### 6.7 Seqnum Ordering Guarantee

The Transaction Tag Allocator guarantees monotonically increasing seqnums per AXID. This property underpins the correctness of the ROB head-pointer ordering scheme. Seqnum allocation stalls if all 4 outstanding slots for an AXID are occupied, providing implicit backpressure on the request path.

### 6.8 N_CLIENTS Scalability

`N_CLIENTS` is a compile-time parameter. AR and AW metadata FIFO depths scale as `4` $\times$ `N_CLIENTS`, ensuring each client always has 4 guaranteed outstanding slots. Idle clients retain their allocated slots; unutilized capacity is not reclaimed dynamically in this revision. A future CIF revision will

introduce a shared slot pool — with 2 guaranteed slots per client and the remaining `(total_depth - 2 × N_CLIENTS)` slots dynamically allocated — controlled by a QoS-aware arbitration fabric.

# 7 Interface Signal Table

This section enumerates every signal crossing a block boundary within the CIF. Signals are grouped by boundary in pipeline order: AXI slave interface first, then write path, then read path, then shared blocks, then the CIF–MC Core interface.

Handshake types used below:

- **VALID/READY** — standard AXI4 handshake; transfer occurs when both asserted.
- **VALID/READY (tied)** — READY permanently high; transfer occurs on VALID.
- **Pulse** — single-cycle strobe, no handshake.
- **Level** — combinational or registered level signal, no handshake.
- **FIFO push/pop** — implicit handshake via FIFO full/empty flags.

## 7.1 AXI Slave Interface (AXI Master → CIF)

| Signal | W | Dir | From | To | Handshake | Description |
|---|---|---|---|---|---|---|
| AWID[5:0] | 6 | in | AXI Master | AW Meta FIFO | VALID/READY | Write transaction ID |
| AWADDR[31:0] | 32 | in | AXI Master | AW Meta FIFO | VALID/READY | Write address |
| AWLEN[7:0] | 8 | in | AXI Master | AW Meta FIFO | VALID/READY | Burst length minus 1 |
| AWSIZE[2:0] | 3 | in | AXI Master | AW Meta FIFO | VALID/READY | Transfer size |
| AWBURST[1:0] | 2 | in | AXI Master | AW Meta FIFO | VALID/READY | Burst type (INCR/WRAP) |
| AWVALID | 1 | in | AXI Master | AW Meta FIFO | VALID/READY | AW channel valid |
| AWREADY | 1 | out | AW Meta FIFO | AXI Master | VALID/READY | AW channel ready; deasserted when client slot full |
| WDATA[511:0] | 512 | in | AXI Master | W Data FIFO | VALID/READY | Write data |
| WSTRB[63:0] | 64 | in | AXI Master | W Data FIFO | VALID/READY | Write byte strobes |
| WLAST | 1 | in | AXI Master | W Data FIFO | VALID/READY | Last beat of burst |
| WVALID | 1 | in | AXI Master | W Data FIFO | VALID/READY | W channel valid |
| WREADY | 1 | out | W Data FIFO | AXI Master | VALID/READY | W channel ready; deasserted when FIFO full |
| BVALID | 1 | out | B Resp Gen | AXI Master | VALID/READY | B channel valid |
| BREADY | 1 | in | AXI Master | B Resp Gen | VALID/READY | B channel ready |
| BRESP[1:0] | 2 | out | B Resp Gen | AXI Master | VALID/READY | Write response (OKAY/DECERR) |
| BID[5:0] | 6 | out | B Resp Gen | AXI Master | VALID/READY | Write response ID |
| ARID[5:0] | 6 | in | AXI Master | AR Meta FIFO | VALID/READY | Read transaction ID |

| | | | | | | |
|---|---|---|---|---|---|---|
| ARADDR[31:0] | 32 | in | AXI Master | AR Meta FIFO | VALID/READY | Read address |
| ARLEN[7:0] | 8 | in | AXI Master | AR Meta FIFO | VALID/READY | Burst length minus 1 |
| ARSIZE[2:0] | 3 | in | AXI Master | AR Meta FIFO | VALID/READY | Transfer size |
| ARBURST[1:0] | 2 | in | AXI Master | AR Meta FIFO | VALID/READY | Burst type (INCR/WRAP) |
| ARVALID | 1 | in | AXI Master | AR Meta FIFO | VALID/READY | AR channel valid |
| ARREADY | 1 | out | AR Meta FIFO | AXI Master | VALID/READY | AR channel ready; deasserted when client slot full |
| RVALID | 1 | out | R Resp Gen | AXI Master | VALID/READY | R channel valid |
| RREADY | 1 | in | AXI Master | R Resp Gen | VALID/READY | R channel ready |
| RDATA[511:0] | 512 | out | R Resp Gen | AXI Master | VALID/READY | Read data |
| RRESP[1:0] | 2 | out | R Resp Gen | AXI Master | VALID/READY | Read response (OKAY/DECERR) |
| RID[5:0] | 6 | out | R Resp Gen | AXI Master | VALID/READY | Read response ID |
| RLAST | 1 | out | R Resp Gen | AXI Master | VALID/READY | Last beat of read burst |

### 7.2 Write Path: AW Metadata FIFO → Request Validator

| Signal | W | Dir | From | To | Handshake | Description |
|---|---|---|---|---|---|---|
| AWADDR[31:0] | 32 | out | AW Meta FIFO | Req Validator | FIFO pop | Write address from FIFO head |
| AWID[5:0] | 6 | out | AW Meta FIFO | Req Validator | FIFO pop | AXI ID from FIFO head |
| AWLEN[7:0] | 8 | out | AW Meta FIFO | Req Validator | FIFO pop | Burst length from FIFO head |
| AWSIZE[2:0] | 3 | out | AW Meta FIFO | Req Validator | FIFO pop | Transfer size from FIFO head |
| AWBURST[1:0] | 2 | out | AW Meta FIFO | Req Validator | FIFO pop | Burst type from FIFO head |

| | | | | | | |
|---|---|---|---|---|---|---|
| STALL | 1 | in | Tag Allocator | Req Validator | Level | Per-AXID stall; Req Validator holds pop when asserted |

### 7.3 Write Path: Request Validator → Tag Allocator / Error Handler

| Signal | W | Dir | From | To | Handshake | Description |
|---|---|---|---|---|---|---|
| REQ_VALID | 1 | out | Req Validator | Tag Allocator | Pulse | Valid transaction; address check passed |
| AXID[5:0] | 6 | out | Req Validator | Tag Allocator | Pulse | AXI ID of valid request |
| AWLEN[7:0] | 8 | out | Req Validator | Tag Allocator | Pulse | Burst length of valid request |
| ERR_VALID | 1 | out | Req Validator | Error Handler | Pulse | Address error notification |
| AXID[5:0] | 6 | out | Req Validator | Error Handler | Pulse | AXI ID preserved for NOP tag allocation |

### 7.4 Write Path: Error Handler → Tag Allocator

| Signal | W | Dir | From | To | Handshake | Description |
|---|---|---|---|---|---|---|
| REQ_VALID | 1 | out | Error Handler | Tag Allocator | Pulse | NOP allocation request; pkt_type pre-set to Err_write |
| AXID[5:0] | 6 | out | Error Handler | Tag Allocator | Pulse | AXI ID for NOP tag allocation |

### 7.5 Shared: Tag Allocator → Burst Splitter

| Signal | W | Dir | From | To | Handshake | Description |
|---|---|---|---|---|---|---|
| TXN_TAG[7:0] | 8 | out | Tag Allocator | Burst Splitter | Pulse | Allocated tag; [7:2]=AXID, [1:0]=seqnum |

| | | | | | | |
|---|---|---|---|---|---|---|
| TAG_VALID | 1 | out | Tag Allocator | Burst Splitter | Pulse | Tag output valid; registered, valid cycle after alloc_en |
| STALL | 1 | out | Tag Allocator | Req Validator / Error Handler | Level | Per-AXID stall; asserted when occupancy[AXID]==4 |

### 7.6 Shared: Tag Allocator → Response Tracker / RTT

| Signal | W | Dir | From | To | Handshake | Description |
|---|---|---|---|---|---|---|
| TXN_TAG[7:0] | 8 | out | Tag Allocator | Resp Tracker | Pulse | Allocated write tag; indexes Response Tracker register file |
| AWLEN[7:0] | 8 | out | Tag Allocator | Resp Tracker | Pulse | Write burst length; exp_pkts = AWLEN+1 |
| TAG_VALID | 1 | out | Tag Allocator | Resp Tracker | Pulse | Allocation strobe to Response Tracker |
| TXN_TAG[7:0] | 8 | out | Tag Allocator | RTT | Pulse | Allocated read tag; indexes RTT register file |
| ARLEN[7:0] | 8 | out | Tag Allocator | RTT | Pulse | Read burst length; exp_pkts = ARLEN+1 |
| TAG_VALID | 1 | out | Tag Allocator | RTT | Pulse | Allocation strobe to RTT |
| alloc_req | 1 | out | Tag Allocator | ROB | Pulse | ROB slot allocation request, coincident with read TAG_VALID |
| AXID[5:0] | 6 | out | Tag Allocator | ROB | Pulse | AXI ID for ROB slot allocation |
| seqnum[1:0] | 2 | out | Tag Allocator | ROB | Pulse | Seqnum for ROB lookup table write |

### 7.7 Shared: Burst Splitter → Packet Formation Unit (Write)

| Signal | W | Dir | From | To | Handshake | Description |
|---|---|---|---|---|---|---|
| pkt_valid | 1 | out | Burst Splitter | Write PFU | VALID/READY | Normalized beat context valid |
| pkt_ready | 1 | in | Write PFU | Burst Splitter | VALID/READY | Downstream ready; holds Emit FSM when low |
| addr[31:0] | 32 | out | Burst Splitter | Write PFU | VALID/READY | Per-beat write address |
| TXN_TAG[7:0] | 8 | out | Burst Splitter | Write PFU | VALID/READY | Transaction tag |
| pkt_type[1:0] | 2 | out | Burst Splitter | Write PFU | VALID/READY | 00=Write, 10=Err_write; carried unchanged from Stage 1 |
| last_beat | 1 | out | Burst Splitter | Write PFU | VALID/READY | Final beat of sub-transaction |
| s1_busy | 1 | out | Burst Splitter | AW Meta FIFO | Level | Stalls Stage 1 FIFO pop during EMIT B state |



### 7.8 Write Path: W Data FIFO → Packet Formation Unit

| Signal | W | Dir | From | To | Handshake | Description |
|---|---|---|---|---|---|---|
| WDATA[511:0] | 512 | out | W Data FIFO | Write PFU | FIFO pop | Write data beat |
| WSTRB[63:0] | 64 | out | W Data FIFO | Write PFU | FIFO pop | Byte enable strobes |
| WLAST | 1 | out | W Data FIFO | Write PFU | FIFO pop | Last beat flag |

### 7.9 Write Path: Packet Formation Unit → Write Packet FIFO

| Signal | W | Dir | From | To | Handshake | Description |
|---|---|---|---|---|---|---|
| PKT_DATA[617:0] | 618 | out | Write PFU | Write Pkt FIFO | FIFO push | Serialized 618-bit write packet: {wrapper, data, addr, txn_tag, pkt_type} |

| | | | | | | |
|---|---|---|---|---|---|---|
| PKT_VALID | 1 | out | Write PFU | Write Pkt FIFO | FIFO push | Packet push strobe |

### 7.10 Write Path: Write Packet FIFO → MC Core

| Signal | W | Dir | From | To | Handshake | Description |
|---|---|---|---|---|---|---|
| PKT_DATA[617:0] | 618 | out | Write Pkt FIFO | MC Core | VALID/READY | Serialized write packet |
| PKT_VALID | 1 | out | Write Pkt FIFO | MC Core | VALID/READY | Packet valid |
| PKT_READY | 1 | in | MC Core | Write Pkt FIFO | VALID/READY | MC Core ready to accept; backpressure propagates upstream |

### 7.11 Write Path: Response Tracker → B Response Generator

| Signal | W | Dir | From | To | Handshake | Description |
|---|---|---|---|---|---|---|
| BRESP_VALID | 1 | out | Resp Tracker | B Resp Gen | Pulse | BRESP ready to emit; asserted cycle after txn_done evaluated |
| BID[5:0] | 6 | out | Resp Tracker | B Resp Gen | Pulse | AXI ID recovered from head_tag[7:2] |
| BRESP[1:0] | 2 | out | Resp Tracker | B Resp Gen | Pulse | OKAY if err_sticky==0; DECERR if err_sticky==1 |

### 7.12 Write Path: Response Tracker → Tag Allocator (Free)

| Signal | W | Dir | From | To | Handshake | Description |
|---|---|---|---|---|---|---|
| WR_FREE_VALID | 1 | out | Resp Tracker | Tag Allocator | Pulse | Write slot free strobe; coincident with BRESP emission |
| WR_FREE_TAG[7:0] | 8 | out | Resp Tracker | Tag Allocator | Pulse | Tag being freed; [7:2]=AXID for occupancy decrement |

### 7.13 Read Path: AR Metadata FIFO → Request Validator

| Signal | W | Dir | From | To | Handshake | Description |
|---|---|---|---|---|---|---|
| ARADDR[31:0] | 32 | out | AR Meta FIFO | Req Validator | FIFO pop | Read address from FIFO head |
| ARID[5:0] | 6 | out | AR Meta FIFO | Req Validator | FIFO pop | AXI ID from FIFO head |
| ARLEN[7:0] | 8 | out | AR Meta FIFO | Req Validator | FIFO pop | Burst length from FIFO head |
| ARSIZE[2:0] | 3 | out | AR Meta FIFO | Req Validator | FIFO pop | Transfer size from FIFO head |
| ARBURST[1:0] | 2 | out | AR Meta FIFO | Req Validator | FIFO pop | Burst type from FIFO head |
| STALL | 1 | in | Tag Allocator | Req Validator | Level | Per-AXID stall; Req Validator holds pop when asserted |



### 7.14 Read Path: Request Validator → Tag Allocator / Error Handler

| Signal | W | Dir | From | To | Handshake | Description |
|---|---|---|---|---|---|---|
| REQ_VALID | 1 | out | Req Validator | Tag Allocator | Pulse | Valid read request; address check passed |
| AXID[5:0] | 6 | out | Req Validator | Tag Allocator | Pulse | AXI ID of valid read request |
| ARLEN[7:0] | 8 | out | Req Validator | Tag Allocator | Pulse | Burst length of valid read request |
| ERR_VALID | 1 | out | Req Validator | Error Handler | Pulse | Address error notification |
| AXID[5:0] | 6 | out | Req Validator | Error Handler | Pulse | AXI ID preserved for NOP tag allocation |

### 7.15 Read Path: Error Handler → Tag Allocator

| Signal | W | Dir | From | To | Handshake | Description |
|---|---|---|---|---|---|---|
| REQ_VALID | 1 | out | Error Handler | Tag Allocator | Pulse | NOP allocation request; pkt_type pre-set to Err_read |
| AXID[5:0] | 6 | out | Error Handler | Tag Allocator | Pulse | AXI ID for NOP tag allocation |

### 7.16 Shared: Burst Splitter → Packet Formation Unit (Read)

| Signal | W | Dir | From | To | Handshake | Description |
|---|---|---|---|---|---|---|
| pkt_valid | 1 | out | Burst Splitter | Read PFU | VALID/READY | Normalized beat context valid |
| pkt_ready | 1 | in | Read PFU | Burst Splitter | VALID/READY | Downstream ready |
| addr[31:0] | 32 | out | Burst Splitter | Read PFU | VALID/READY | Per-beat read address |
| TXN_TAG[7:0] | 8 | out | Burst Splitter | Read PFU | VALID/READY | Transaction tag |
| pkt_type[1:0] | 2 | out | Burst Splitter | Read PFU | VALID/READY | 01=Read, 11=Err_read |
| last_beat | 1 | out | Burst Splitter | Read PFU | VALID/READY | Final beat of sub-transaction |
| s1_busy | 1 | out | Burst Splitter | AR Meta FIFO | Level | Stalls Stage 1 FIFO pop during EMIT B state |

### 7.17 Read Path: Packet Formation Unit → Read Packet FIFO

| Signal | W | Dir | From | To | Handshake | Description |
|---|---|---|---|---|---|---|
| PKT_DATA[41:0] | 42 | out | Read PFU | Read Pkt FIFO | FIFO push | Serialized 42-bit read packet: {addr, txn_tag, pkt_type} |

| | | | | | | |
|---|---|---|---|---|---|---|
| PKT_VALID | 1 | out | Read PFU | Read Pkt FIFO | FIFO push | Packet push strobe |

### 7.18 Read Path: Read Packet FIFO → MC Core

| Signal | W | Dir | From | To | Handshake | Description |
|---|---|---|---|---|---|---|
| PKT_DATA[41:0] | 42 | out | Read Pkt FIFO | MC Core | VALID/READY | Serialized read request packet |
| PKT_VALID | 1 | out | Read Pkt FIFO | MC Core | VALID/READY | Packet valid |
| PKT_READY | 1 | in | MC Core | Read Pkt FIFO | VALID/READY | MC Core ready; deasserted by ROB watermark |

### 7.19 MC Core Completion Interface (MC Core → CIF)

| Signal | W | Dir | From | To | Handshake | Description |
|---|---|---|---|---|---|---|
| RESP_VALID | 1 | in | MC Core | ROB / Resp Tracker / RTT | VALID/READY (tied) | Completion valid |
| RESP_READY | 1 | out | CIF | MC Core | VALID/READY (tied) | Tied permanently high; CIF never stalls MC Core completions |
| resp_type[1:0] | 2 | in | MC Core | ROB / Resp Tracker / RTT | VALID/READY (tied) | 00=RD_DATA (→ROB, RTT), 01=WR_ACK (→Resp Tracker), 10=ERR |
| tag[7:0] | 8 | in | MC Core | ROB / Resp Tracker / RTT | VALID/READY (tied) | Completion tag; [7:2]=AXID, [1:0]=seqnum |
| data[511:0] | 512 | in | MC Core | ROB | VALID/READY (tied) | Read data; valid only when resp_type==00 |
| status[3:0] | 4 | in | MC Core | ROB / Resp Tracker | VALID/READY (tied) | Status/error code; encoding TBD with MC team |

### 7.20 Read Path: ROB → Merge Logic

| Signal | W | Dir | From | To | Handshake | Description |
|---|---|---|---|---|---|---|
| data[511:0] | 512 | out | ROB | Merge Logic | VALID/READY | In-order read data beat emitted by Emit Logic |
| status[3:0] | 4 | out | ROB | Merge Logic | VALID/READY | Status for emitted beat |
| AXID[5:0] | 6 | out | ROB | Merge Logic | VALID/READY | AXID of emitted beat |
| rob_last | 1 | out | RTT (via ROB) | Merge Logic | VALID/READY | Final beat of transaction; sourced from RTT, passed through ROB coincident with final data beat |

### 7.21 Read Path: RTT → ROB / Merge Logic

| Signal | W | Dir | From | To | Handshake | Description |
|---|---|---|---|---|---|---|
| rob_last | 1 | out | RTT | ROB / Merge Logic | Pulse | One-cycle pulse on final MC Core completion for a transaction; cmp_pkts == exp_pkts |
| RD_FREE_VALID | 1 | out | RTT | Tag Allocator | Pulse | Read slot free strobe; coincident with rob_last |
| RD_FREE_TAG[7:0] | 8 | out | RTT | Tag Allocator | Pulse | Tag being freed; [7:2]=AXID for occupancy decrement |

### 7.22 Read Path: Merge Logic → R Response Generator

| Signal | W | Dir | From | To | Handshake | Description |
|---|---|---|---|---|---|---|
| RDATA[511:0] | 512 | out | Merge Logic | R Resp Gen | VALID/READY | Ordered read data beat |
| RID[5:0] | 6 | out | Merge Logic | R Resp Gen | VALID/READY | AXI ID recovered from txn_tag[7:2] |
| status[3:0] | 4 | out | Merge Logic | R Resp Gen | VALID/READY | Beat status; drives DECERR when error |
| RLAST | 1 | out | Merge Logic | R Resp Gen | VALID/READY | Last beat flag derived from rob_last |

### 7.23 ROB Backpressure Control

| Signal | W | Dir | From | To | Handshake | Description |
|---|---|---|---|---|---|---|
| watermark_hit | 1 | out | ROB Occ Counter | PKT_READY ctrl | Level | Asserted when occupancy $\geq 38$; combinational from counter |
| PKT_READY | 1 | out | ROB | Read Pkt FIFO | Level | Deasserted when watermark_hit high; halts new read requests to MC Core |

# 8 Arbitration Policy

## 8.1 Overview

Two blocks in CIF are shared across all $N\_CLIENTS = 32$ AXI clients: the **Transaction Tag Allocator** and the **Burst Splitter**. Both sit on the common request path after the per-path Request Validators. This section defines how access to these shared resources is arbitrated.

No other block requires explicit arbitration. The AR and AW Metadata FIFOs are partitioned statically (4 slots per client); the ROB, RTT, and Response Tracker are indexed directly by `txn_tag` and require no arbitration. The completion path from MC Core is a single bus fanned out to the ROB, RTT, and Response Tracker simultaneously, with each block filtering on `resp_type` — no arbiter required.

## 8.2 Transaction Tag Allocator

The Tag Allocator is a single shared instance serving both read and write paths.

### Arbitration Mechanism

The Tag Allocator accepts exactly one allocation request per cycle. Upstream priority is resolved as follows:

1. **Write path** and **read path** requests arrive at the Tag Allocator from their respective Request Validators (or Error Handlers). In CIF v1, write and read paths share the Burst Splitter sequentially (single pipeline); the upstream MUX feeding the Tag Allocator selects between write-path and read-path requests using **fixed priority, write > read**. If both paths present a request in the same cycle, the write-path request is accepted; the read-path request is held for the next cycle.
2. Within a single path, only one AXID is presented per cycle (the FIFO head entry). No further arbitration is required at this level.

### Per-AXID Stall

Allocation for a given AXID is blocked when `occupancy[AXID] == MAX_OUTSTANDING` (4). The Stall Logic asserts `STALL` combinationally to the upstream block for that specific AXID only. Other AXIDs are unaffected. `STALL` is released in the cycle after the occupancy counter decrements (i.e., cycle $N + 1$ after `WR_FREE_VALID` or `RD_FREE_VALID` fires for a tag belonging to that AXID).

### Error Path

The Error Handler presents allocation requests using the same interface as the Request Validator (`REQ_VALID`, `AXID`). Error-path allocations are subject to the same per-AXID stall and write/read priority rules. No bypass or priority elevation is granted to error transactions.

### Simultaneous Free and Allocate

If `alloc_en` and `dec_occ` fire in the same cycle for the same AXID (one slot freed while a new one is being allocated), the occupancy counter holds constant (net-zero change). This is a valid steady-state operating condition and requires no special handling.

## 8.3 Burst Splitter

The Burst Splitter is a single shared two-stage pipeline instance serving both read and write paths across all clients.

### Arbitration Mechanism

Access to the Burst Splitter is implicitly arbitrated by the upstream MUX that feeds the Tag Allocator. Since the Tag Allocator and Burst Splitter are driven from the same upstream MUX with the same priority policy (write > read, one request per cycle), the Burst Splitter processes exactly one transaction at a time.

### Pipeline Stall Policy

The Burst Splitter Emit FSM stalls Stage 1 (via `s1_busy`) during EMIT B to prevent the next FIFO entry from being consumed while the second sub-transaction of a split is being emitted. `s1_busy` is deasserted on the cycle the FSM returns to IDLE. Backpressure from downstream (`pkt_ready = 0`) holds the FSM in its current state; `s1_busy` remains asserted until IDLE is reached.

### Tag Allocator Stall Folding

If the Tag Allocator asserts `STALL` before Stage 1 can proceed, the stall is folded into `s1_busy`. Stage 1 does not advance until `STALL` deasserts and a valid tag is available.

### Throughput

Under no-split, no-stall conditions the Burst Splitter achieves one transaction per cycle. A row-boundary split costs one additional cycle per transaction (EMIT A + EMIT B). WRAP bursts iterate over the pre-computed `beat_addrs[]` table; throughput is one beat per cycle per WRAP address, plus one additional cycle if a row-boundary split is also required.

# 9 Reset and Initialization Sequence

## 9.1 Overview

CIF supports two reset modes:

- **Power-on reset (POR):** Asserted by the FPGA fabric on configuration. All flip-flops and BRAM init values take effect. CIF remains in reset until the AXI interconnect and MC Core are both ready.
- **Soft reset:** Driven by a register write from a management plane. Must drain the pipeline before asserting; returns CIF to the same state as POR without reconfiguring the FPGA.

All resets are **synchronous, active-high**, qualified on `posedge clk`. There is no asynchronous reset path in CIF.

## 9.2 Reset Signal

| Signal | W | Source | Description |
|---|---|---|---|
| `rst_n` | 1 | FPGA top / mgmt plane | Active-low synchronous reset; asserted low to reset all CIF blocks |

## 9.3 Power-On Reset Sequence

The following sequence describes the required initialization order. Steps within the same numbered stage may proceed in parallel.

1. **Assert `rst_n = 0`.** All registered state is cleared synchronously on the next `posedge clk`:
   - AR/AW Metadata FIFO and W Data FIFO: empty, `ARREADY / AWREADY / WREADY = 0`.
   - Write and Read Packet FIFOs: empty.
   - Tag Allocator: all 64 occupancy counters ← 0; all 64 seqnum counters ← 0.
   - Response Tracker register file: all entries cleared (`cmp_pkts = 0`, `err_sticky = 0`, `exp_pkts = 0`).
   - Response Tracker head pointer array: all 64 entries ← 0.
   - RTT register file: all 256 entries cleared.
   - ROB BRAM: all 64 slots `valid = 0`.
   - ROB free list: all 64 bits ← 1 (all slots free).
   - ROB lookup table: all 256 entries ← 0.
   - ROB head pointer array: all 64 registers ← 0.
   - ROB occupancy counter ← 0.
   - Burst Splitter Emit FSM ← IDLE; all internal state registers cleared.
   - Request Validators: combinational; no state to clear.

- Error Handlers: no persistent state to clear.
- `RESP_READY` driven high immediately (tied; no reset dependency).

2. **Wait for MC Core ready.** CIF must not issue any packets until MC Core signals it is ready to accept traffic. `PKT_VALID` must be held low (Packet FIFOs are empty during reset; no additional gating required if FIFOs reset correctly).
3. **Deassert `rst_n = 1`.** On the first `posedge clk` after deassertion:
   - `AWREADY` and `ARREADY` are driven based on per-client FIFO slot availability. Both are high immediately after reset since all FIFO slots are free.
   - `WREADY` is driven high immediately (W Data FIFO empty).
   - The Tag Allocator begins accepting allocation requests.
   - The Burst Splitter Emit FSM begins processing from IDLE.
4. **AXI traffic may begin.** First valid AXI transaction is accepted on the cycle after `AWREADY / ARREADY` are high and the AXI master presents a valid request.

### 9.4 Soft Reset Sequence

Soft reset must be preceded by a pipeline drain to avoid losing in-flight transactions.

1. **Quiesce upstream.** The management plane instructs all AXI masters to stop issuing new transactions. New `AWVALID` and `ARVALID` are withheld.
2. **Drain the request pipeline.** Wait until:
   - AR and AW Metadata FIFOs are empty.
   - W Data FIFO is empty.
   - Write and Read Packet FIFOs are empty.
   - Burst Splitter Emit FSM is in IDLE.
   - Tag Allocator: all 64 occupancy counters = 0 (no in-flight transactions).
3. **Drain the completion pipeline.** Wait until:
   - ROB occupancy counter = 0 (all read completions have been emitted to Merge Logic).
   - Response Tracker: all 256 register file entries clear (`cmp_pkts == exp_pkts` and freed, or all `exp_pkts == 0`).
   - RTT: all 256 entries clear.
   - No pending `BRESP` or `RRESP` outstanding on the AXI B or R channels.
4. **Assert `rst_n = 0`.** Proceed identically to steps 1–4 of the power-on reset sequence.

### 9.5 BRAM Initialization Note

The ROB BRAM `valid` bits are the only BRAM state that must be explicitly initialized. On the cycle `rst_n` deasserts, all 64 `valid` bits must read as 0. This is achieved by:

- Using a synchronous-reset flip-flop for each `valid` bit (implemented as distributed RAM or shadow registers alongside the BRAM data), **or**
- Performing a 64-cycle initialization write loop on the BRAM write port during reset, setting `valid = 0` at each address before `rst_n` deasserts.

The second option requires holding `rst_n` low for at least 64 cycles after POR. This is the recommended approach for UltraScale BRAM inference.

# 10 Error Propagation Timing

## 10.1 Overview

Address errors detected by the Request Validator are propagated through the normal CIF pipeline as NOP packets using `pkt_type[1:0] = 10` (`Err_write`) or `11` (`Err_read`). This section traces the cycle-by-cycle timing of `pkt_type` from error detection to AXI error response, and identifies where `pkt_type` is set, carried, and consumed. Timing counts assume no stalls unless noted.

### 10.2 Write Path Error Propagation

| Cycle | Stage | Event |
|---|---|---|
| $N$ | Request Validator | Address Range Check evaluates `addr_ok = 0` combinationally. `ERR_VALID` asserted to Error Handler; `AXID` forwarded. AW Metadata FIFO entry is consumed (popped) in this cycle. |
| $N$ | Error Handler | Error Handler sets `pkt_type = Err_write` and presents `REQ_VALID` + `AXID` to Tag Allocator in the same cycle. |
| $N+1$ | Tag Allocator | `alloc_en` fires (assuming no `STALL`). `TXN_TAG` and `TAG_VALID` registered; valid on cycle $N+1$. Occupancy counter and seqnum counter update take effect on cycle $N+1$. |
| $N+1$ | Burst Splitter Stage 1 | `pkt_type = Err_write` enters Stage 1 alongside `TXN_TAG`, `ADDR`, `AXLEN`, `AXSIZE`. Stage 1 computes split decision combinationally; pipeline register clocked on cycle $N+1$. |
| $N+2$ | Burst Splitter Stage 2 | Emit FSM enters EMIT A. `pkt_type` carried unchanged through Sub-txn A mux. Both sub-transactions (if split) carry the same `pkt_type = Err_write`. |
| $N+2$ | Packet Formation Unit | Write PFU assembles 618-bit packet with `pkt_type[1:0] = 10` in bits [1:0]. `data[511:0]` and `wrapper[63:0]` are taken from the W Data FIFO as-is; no drain or discard. |
| $N+2$ | Write Packet FIFO | Packet pushed into FIFO. |
| $N+2+F_W$ | MC Core | `Err_write` packet dequeued from Write Packet FIFO and issued to MC Core (`PKT_VALID` asserted). $F_W$ = Write Packet FIFO occupancy delay (0 under no backpressure). MC Core discards write data; returns error completion (`resp_type = 10`) on the completion bus. |
| $N+2+F_W+L_{MC}$ | Response Tracker | MC Core asserts `RESP_VALID` with `resp_type = 10`. Completion Update Logic increments `cmp_pkts` and sets `err_sticky = 1`. Compare Logic evaluates `cmp_pkts == exp_pkts` combinationally (true on this cycle for a single-packet error transaction). `txn_done` pulses. |
| $N+3+F_W+L_{MC}$ | B Response Generator | Emit Logic (registered) drives `BRESP_VALID`, `BID`, and `BRESP = DECERR` to B Response Generator on the cycle after `txn_done`. `BVALID` is asserted on the AXI B channel. |

$L_{MC}$ denotes MC Core's internal latency from packet receipt to error completion assertion. This is a MC Core parameter and is not constrained by CIF.

**Key invariant:** `pkt_type` is set once (at the Error Handler output, cycle $N$) and carried unchanged through the S1→S2 pipeline register and Sub-txn muxes. Neither the Burst Splitter nor the PFU inspect or modify `pkt_type`. Both sub-transactions of a split carry the same `pkt_type`.

### 10.3 Read Path Error Propagation

| Cycle | Stage | Event |
|---|---|---|
| $N$ | Request Validator | `addr_ok = 0` combinationally. `ERR_VALID` asserted to Error Handler; `AXID` forwarded. AR Metadata FIFO entry consumed. |
| $N$ | Error Handler | `pkt_type = Err_read` set. `REQ_VALID` + `AXID` presented to Tag Allocator. |
| $N+1$ | Tag Allocator | `TXN_TAG` and `TAG_VALID` registered and valid. ROB slot allocated via `alloc_req`. RTT entry allocated via `TAG_VALID` path with `exp_pkts = ARLEN+1`. |
| $N+1$ | Burst Splitter Stage 1 | `pkt_type = Err_read` enters Stage 1; pipeline register clocked. |
| $N+2$ | Burst Splitter Stage 2 | Emit FSM emits beat(s) with `pkt_type = Err_read` unchanged. |
| $N+2$ | Packet Formation Unit | Read PFU assembles 42-bit packet with `pkt_type[1:0] = 11` in bits [1:0]. No data field in read packets. |
| $N+2$ | Read Packet FIFO | Packet pushed into FIFO. |
| $N+2+F_R$ | MC Core | `Err_read` packet issued to MC Core. MC Core returns error completion (`resp_type = 10`) with no data. |
| $N+2+F_R+L_{MC}$ | ROB | Completion Input writes `status` and `valid = 1` to the allocated ROB slot with error status preserved. RTT: `cmp_pkts` incremented; `rob_last` pulses (single-beat transaction). |
| $N+2+F_R+L_{MC}$ | Merge Logic | Emit Logic emits slot to Merge Logic with `status` indicating error. `rob_last` forwarded simultaneously. |
| $N+3+F_R+L_{MC}$ | R Response Generator | `RVALID` asserted with `RRESP = DECERR`, `RID` recovered from `tag[7:2]`, `RLAST = 1`. |

### 10.4 Split Transaction Error Propagation

When an error transaction also requires a row-boundary split (`split_needed = 1`), both sub-transactions carry the same `pkt_type`. The implications are:

- **Write path:** Both `Err_write` packets are issued to MC Core. MC Core returns one error completion per packet. The Response Tracker increments `cmp_pkts` on each completion; `err_sticky` is set on the first. `txn_done` fires only after both completions arrive (`cmp_pkts == exp_pkts`). `BRESP = DECERR` is emitted once, covering the entire transaction.
- **Read path:** Both `Err_read` packets are issued to MC Core. MC Core returns one error completion per packet (no data). The RTT increments `cmp_pkts` on each; `rob_last` fires after the second. The ROB emits two slots (both with error status); Merge Logic forwards both

to the R Response Generator. `RLAST` is asserted on the final beat; both beats carry `RRESP = DECERR`.

- **Cycle cost of split:** EMIT B adds one cycle to the Burst Splitter stage (Stage 2 holds `s1_busy` for one additional cycle). All downstream stages (PFU, FIFO, MC Core, response path) are otherwise unaffected by the split.

### 10.5 Err_write W Beat Handling

W beats for an `Err_write` transaction flow through the W Data FIFO and Write PFU without modification or draining. The `pkt_type` field in the assembled packet signals to MC Core that the data is invalid; MC Core discards the payload. No special W-beat gating or bypass path exists in CIF. The number of W beats consumed by the PFU is determined by `AWLEN`, consistent with the normal write path.

### 10.6 Timing Summary

| Stage | Write Error Latency | Read Error Latency |
|---|---|---|
| Req Validator → Error Handler | 0 (combinational) | 0 (combinational) |
| Error Handler → Tag Allocator output | 1 cycle | 1 cycle |
| Tag Allocator → Burst Splitter S2 output | 1 cycle | 1 cycle |
| Burst Splitter S2 → Packet FIFO push | 0 (same cycle) | 0 (same cycle) |
| Packet FIFO → MC Core (no backpressure) | 0 | 0 |
| MC Core latency | $L_{MC}$ | $L_{MC}$ |
| Final completion → BRESP/RRESP (no split) | 1 cycle (Emit Logic reg.) | 0 (ROB emit + Merge Logic) |
| **Total CIF-internal latency (no split, no stall)** | **$3 + L_{MC}$ cycles** | **$2 + L_{MC}$ cycles** |

## A Burst Splitter

### A.1 Overview

The Burst Splitter is a two-stage pipelined block shared across all `N_CLIENTS` AXI clients. It sits between the AW/AR FIFO and the Packet Formation Unit. Its function is to convert validated AXI burst requests into normalized per-beat packets, splitting bursts that cross DRAM row boundaries into two sub-transactions.

**Input:** AW/AR FIFO output — `ADDR`, `AXID`, `AXLEN`, `AXSIZE`, `AXBURST`

**Output:** Per-beat context (`addr`, `tag`, `pkt_type`, `beat_cnt`) → Packet Formation Unit via `pkt_valid`

### A.2 Stage 1 — Decode & Compute (1 cycle, registered output)

Stage 1 runs combinatorially and captures all results into the S1→S2 pipeline register on the next clock edge. Three sub-blocks operate in parallel:

### A.3 Burst Type Decoder

- Input: `AXBURST[1:0]` from FIFO
- Output: `is_wrap` / `is_incr` flag → pipeline register
- FIXED burst type is not supported (design-time assertion)

### A.4 Row Boundary Checker

- Inputs: `ADDR`, `AXLEN`, `AXSIZE` from FIFO
- Computes: `addr_start + burst_bytes` vs `ROW_MASK`
- Outputs: `split_needed` flag, `row_boundary` address → Split Point Calc and Beat Count Calc

### A.5 WRAP Address Generator

- Inputs: `ADDR`, `AXLEN`, `AXSIZE` from FIFO
- Active only when `is_wrap = 1`
- Outputs: `wrap_boundary`, `beat_addrs[0..N]` → pipeline register
- Maximum 16 entries (AXI4 WRAP burst max `AXLEN` = 15)

### A.6 Split Point Calc

- Inputs: `row_boundary` from Row Boundary Checker; `ADDR` from FIFO
- Computes: `addr_b = row_boundary`, `beat_cnt_b = total_beats − beat_cnt_a`
- Output: `addr_b`, `beat_cnt_b` → pipeline register

### A.7 Beat Count Calc

- Inputs: `row_boundary` from Row Boundary Checker; `AXLEN` from FIFO
- Computes: `beat_cnt_a = beats_to_row_end`, `total_beats = AXLEN + 1`
- Output: `beat_cnt_a`, `total_beats` → pipeline register

### A.8 S1→S2 Pipeline Register (clocked)

{ `addr_a, beat_cnt_a, addr_b, beat_cnt_b, split_needed, is_wrap, beat_addrs[], axid, pkt_type, tag` }

### A.9 Stage 2 — Emit & Track

Stage 2 is multi-cycle when a split is needed. It stalls Stage 1 (via `s1_busy`) while emitting sub-transaction B.

### A.10 Emit FSM

- `IDLE` → `EMIT_A`
- `EMIT_A` → `IDLE` (split_needed = 0)
- `EMIT_A` → `EMIT_B` (split_needed = 1)
- `EMIT_B` → `IDLE`
- Hold current state if `pkt_ready = 0`
- Asserts `s1_busy` during `EMIT_B` to stall Stage 1

### A.11 Sub-transaction A Mux

- Selects: addr_a, beat_cnt_a, tag, pkt_type from pipeline register
- Forwards to Pkt Assembler on cycle N

### A.12 Sub-transaction B Mux

- Selects: addr_b, beat_cnt_b, same tag, pkt_type from pipeline register
- Forwards to Pkt Assembler on cycle N+1

### A.13 Beat Tracker

- Maintains beat_cnt_reg; decrements on pkt_accept
- Asserts last_beat flag when count reaches zero
- Output: last_beat → Pkt Assembler

### A.14 WRAP Beat Address Table

- Up to 16-entry register file, written once on S1→S2 pipeline register transfer
- Input: beat_addrs[] from S1→S2 pipeline register
- Read sequentially by beat_idx counter
- Output: current beat address → Pkt Assembler

### A.15 Pkt Assembler

- Inputs: addr, tag, pkt_type, beat_cnt from Sub-txn A/B mux; last_beat from Beat Tracker; beat address from WRAP beat addr table
- Assembles normalized read (42-bit) or write (618-bit) packet
- Asserts pkt_valid → Packet Formation Unit

### A.16 Backpressure & Control Signals

| Signal | Direction | Description |
|---|---|---|
| s1_busy | Emit FSM → AW/AR FIFO | Stalls Stage 1 from consuming next FIFO entry during EMIT_B |
| pkt_ready | Packet Formation Unit → Emit FSM | Back-pressure from downstream; FSM holds state when low |
| pkt_valid | Pkt Assembler → Packet Formation Unit | Indicates normalized packet ready for consumption |

### A.17 Three Paths Through the Splitter

**Path A — No split (INCR, no row crossing)**

split_needed = 0. Stage 2 emits one packet (EMIT_A → IDLE in 1 cycle). s1_busy never asserted. Throughput: 1 transaction / cycle.

**Path B — Row-boundary split (INCR, crosses boundary)**
`split_needed = 1`. Stage 1 computes `addr_b`, `beat_cnt_b`. Stage 2: `EMIT_A` cycle N, asserts `s1_busy`, `EMIT_B` cycle N+1. Throughput: 1 transaction / 2 cycles.

**Path C — WRAP burst**
WRAP address generator fills `beat_addrs[]`. Stage 2 iterates via WRAP beat addr table indexed by `beat_idx`. May also split at row boundary.

### A.18 Internal State

| Field | Description |
|---|---|
| `base_addr` | Base address of current burst |
| `beat_idx` | Current beat index within burst |
| `beats_left` | Remaining beats to emit |
| `burst_type` | INCR or WRAP |
| `burst_len` | Total beats (AXLEN + 1) |
| `txn_tag` | Allocated transaction tag `[AXID[5:0] + seqnum[1:0]]` |

### A.19 Timing Notes

- Stage 1 combinatorial depth must close timing in 1 cycle. WRAP address generator is the critical path — limited to AXLEN $\leq 15$ for WRAP bursts (AXI4 constraint).
- S1→S2 register clocked at `posedge clk`.
- `pkt_ready = 0` holds the Emit FSM in its current state; `s1_busy` remains asserted until `IDLE` is reached.
- Tag Allocator must have a valid tag before Stage 1 can proceed — Tag Allocator stall folds into `s1_busy`.
- Error path (`pkt_type = Err_write / Err_read`): set in Stage 1, carried through pipeline register unchanged. Both sub-transactions on a split carry the same `pkt_type`.

## B Request Validator

### B.1 Overview

The Request Validator is a purely combinational block instantiated once each on the read and write paths. It pops entries from the AR or AW Metadata FIFO, performs a single runtime check (address range validation), and steers the transaction to one of two downstream paths:

- **Valid path:** Transaction forwarded to the Transaction Tag Allocator with `REQ_VALID` asserted.
- **Error path:** Transaction forwarded to the Error Handler with `ERR_VALID` asserted and `AXID` preserved.

The Request Validator contains no storage, no counters, and no state. All protocol-level checks (burst type, WRAP legality, alignment, burst length) are enforced as design-time assertions in the verification environment only. Compliant AXI4 masters are assumed at runtime.

| Parameter | Value | Description |
|---|---|---|
| ADDR_W | 32 | Address width in bits |
| AXID_W | 6 | AXI ID width |
| LEN_W | 8 | Burst length field width (ARLEN/AWLEN) |

Table 33: Request Validator parameters

## B.2 Design Parameters

## B.3 Internal Blocks

## B.4 Address Range Check

- **Type:** Purely combinational.
- **Input:** ADDR[31:0] from the AR/AW Metadata FIFO.
- **Function:** Checks whether ADDR falls within the valid memory address range defined at synthesis time.
- **Output:** addr_ok (1-bit) — asserted when the address is valid, deasserted on an address error.
- **Scope:** Address range check only. No burst type, no alignment, no length checks at runtime.

## B.5 Output Mux

- **Type:** Purely combinational.
- **Function:** Routes the transaction to the correct downstream block based on addr_ok.
- **Valid path** (addr_ok == 1): asserts REQ_VALID to the Tag Allocator; forwards AXID[5:0] and ARLEN/AWLEN[7:0].
- **Error path** (addr_ok == 0): asserts ERR_VALID to the Error Handler; forwards AXID[5:0].
- Only one output path is active per transaction. Both paths are never asserted simultaneously.

## B.6 Interface Signals

## B.7 Inputs

| Signal | Width | Source | Description |
|---|---|---|---|
| ADDR[31:0] | 32 | AR/AW Metadata FIFO | Transaction address |
| AXID[5:0] | 6 | AR/AW Metadata FIFO | AXI ID |
| ARLEN/AWLEN[7:0] | 8 | AR/AW Metadata FIFO | Burst length |
| STALL | 1 | Tag Allocator | Per-AXID stall; holds output until deasserted |

Table 34: Request Validator input signals

## B.8 Outputs

| Signal | Width | Destination | Description |
|---|---|---|---|
| REQ_VALID | 1 | Tag Allocator | Valid transaction request |
| AXID[5:0] | 6 | Tag Allocator | Forwarded AXI ID |
| ARLEN/AWLEN[7:0] | 8 | Tag Allocator | Forwarded burst length |
| ERR_VALID | 1 | Error Handler | Address error notification |
| AXID[5:0] | 6 | Error Handler | AXI ID preserved for NOP tag allocation |

Table 35: Request Validator output signals

## B.9 Functional Behaviour

1. Request Validator pops the head entry from the AR/AW Metadata FIFO: ADDR[31:0], AXID[5:0], ARLEN/AWLEN[7:0].
2. Address Range Check evaluates addr_ok combinationally.
3. If addr_ok == 1 and STALL is deasserted: Output Mux asserts REQ_VALID to the Tag Allocator with AXID and ARLEN/AWLEN.
4. If addr_ok == 0: Output Mux asserts ERR_VALID to the Error Handler with AXID. The Error Handler sets pkt_type = Err_write / Err_read and injects the transaction into the normal pipeline via the Tag Allocator.
5. If STALL is asserted (Tag Allocator full for this AXID): the Request Validator holds its outputs stable and does not pop the next FIFO entry until STALL deasserts.

## B.10 Timing Notes

- Fully combinational. Output (REQ_VALID or ERR_VALID) is valid in the same cycle as the FIFO head entry is presented.
- STALL from the Tag Allocator is also combinational — the combined path (FIFO output → Address Range Check → Output Mux → Tag Allocator Stall Logic → STALL back) is a combinational loop that must be timed carefully. A register stage may be inserted between the Request Validator output and the Tag Allocator input if timing closure requires it, at the cost of one cycle of allocation latency.

## B.11 Edge Cases

- **Error path stall:** An address error transaction still consumes a Tag Allocator slot. If the AXID is at maximum outstanding (STALL asserted), the error path is also held. ERR_VALID is not asserted until STALL deasserts.
- **Protocol violations:** Illegal burst type, unaligned WRAP, and maximum burst length violations are not checked at runtime. They are enforced as design-time assertions in the verification environment. A non-compliant master will cause undefined behaviour.
- **Simultaneous valid and error:** Not possible — addr_ok is a single-bit result; exactly one output path is active per transaction.

# C Transaction Tag Allocator

## C.1 Overview

The Transaction Tag Allocator is a shared combinational/registered block instantiated once in the CIF and used by both the read and write paths across all `N_CLIENTS = 32` AXI clients.

Its responsibilities are:

- Allocate a unique `txn_tag[7:0]` for every validated transaction entering the pipeline.
- Enforce the per-AXID outstanding transaction limit of `MAX_OUTSTANDING = 4`.
- Stall upstream (Request Validator / Error Handler) on a per-AXID basis when the outstanding limit is reached.
- Free an occupied slot when a transaction completes — via `BRESP` on the write path, or `rob_last` on the read path.

Tag encoding (agreed with MC Core team):

```
txn_tag[7:0]:
  [7:2] = AXID[5:0]    — 6-bit AXI ID, up to 64 unique IDs
  [1:0] = seqnum[1:0] — per-AXID slot index, wraps at 4
```

## C.2 Design Parameters

| Parameter | Value | Description |
|---|---|---|
| `AXID_W` | 6 | AXI ID width; 64 unique IDs |
| `MAX_OUTSTANDING` | 4 | Max outstanding txns per AXID |
| `SEQNUM_W` | 2 | Seqnum width ($\log_2$ MAX_OUTSTANDING) |
| `TXN_TAG_W` | 8 | Tag width = AXID_W + SEQNUM_W |
| `N_AXIDS` | 64 | Total AXID space ($2^6$) |
| `OCC_CTR_W` | 2 | Occupancy counter width per AXID |

Table 36: Transaction Tag Allocator parameters

## C.3 Internal Blocks

## C.4 Occupancy Counter Array

- **Structure:** 64 independent 2-bit saturating counters, one per AXID.
- **Increment:** On every successful allocation (`alloc_en` asserted), the counter for the requesting AXID is incremented.
- **Decrement:** On assertion of `WR_FREE_VALID` or `RD_FREE_VALID`, the counter for the AXID extracted from the incoming tag (`tag[7:2]`) is decremented.
- **Read port:** Continuously exposes `occupancy[AXID]` to Stall Logic.

### C.5 Seqnum Counter Array

- **Structure:** 64 independent 2-bit wrap counters, one per AXID. Represents the *next* seqnum to be allocated.
- **Increment:** On every successful allocation (`alloc_en` asserted), the counter for the requesting AXID is incremented modulo 4.
- **Read port:** Continuously exposes `seqnum[1:0]` for the requesting AXID to Allocate Logic.
- **No decrement:** Seqnum counters are allocation-only; they are never decremented. The occupancy counter separately tracks the number of in-flight slots.

### C.6 Stall Logic

- **Type:** Purely combinational.
- **Function:** Asserts `STALL` for a given AXID when `occupancy[AXID] == MAX_OUTSTANDING (4)`.
- **Granularity:** Per-AXID. Only the AXID currently presented by the upstream block is evaluated; other AXIDs are unaffected.
- **Output:** `STALL` fed back to Request Validator and Error Handler.

### C.7 Allocate Logic

- **Gate condition:** Fires only when `REQ_VALID` is asserted and `STALL` is deasserted (`alloc_en = REQ_VALID & ~STALL`).
- **Tag formation:**

  $$\text{TXN\_TAG}[7:0] = \{\ \text{AXID}[5:0],\ \text{seqnum}[1:0]\ \}$$

- **Side-effects on alloc:** `inc_occ` to Occupancy Counter Array; `inc_seq` to Seqnum Counter Array.
- **Output:** `TXN_TAG[7:0]` and `TAG_VALID` forwarded to Burst Splitter.
- **Error path:** Error Handler presents AXID with `REQ_VALID` asserted; allocation proceeds identically. The resulting tag is used to form a NOP packet (`pkt_type = Err_write / Err_read`) in the normal pipeline. No special casing in Allocate Logic.

### C.8 Free Logic

- **Two free ports:**
  - Write path: `WR_FREE_VALID`, `WR_FREE_TAG[7:0]` — sourced from Response Tracker on BRESP emission.
  - Read path: `RD_FREE_VALID`, `RD_FREE_TAG[7:0]` — sourced from Read Transaction Table on `rob_last` assertion.
- **AXID extraction:** `AXID = tag[7:2]` in both cases.
- **Action:** Asserts `dec_occ` to Occupancy Counter Array for the extracted AXID.

- **Simultaneous frees:** Both ports may assert in the same cycle for different AXIDs — handled independently. Simultaneous frees on the same AXID (one write + one read) are not possible by construction (a tag is either a read or a write transaction, never both).

### C.9 Interface Signals

### C.10 Inputs

| Signal | Width | Source | Description |
|---|---|---|---|
| REQ_VALID | 1 | Req Validator / Error Handler | Allocation request valid |
| AXID[5:0] | 6 | Req Validator / Error Handler | Requesting AXI ID |
| WR_FREE_VALID | 1 | Response Tracker | Write path free strobe |
| WR_FREE_TAG[7:0] | 8 | Response Tracker | Tag being freed (write) |
| RD_FREE_VALID | 1 | Read Txn Table | Read path free strobe |
| RD_FREE_TAG[7:0] | 8 | Read Txn Table | Tag being freed (read) |

Table 37: Tag Allocator input signals

### C.11 Outputs

| Signal | Width | Destination | Description |
|---|---|---|---|
| TXN_TAG[7:0] | 8 | Burst Splitter | Allocated transaction tag |
| TAG_VALID | 1 | Burst Splitter | Tag output valid |
| STALL | 1 | Req Validator / Error Handler | Per-AXID stall signal |

Table 38: Tag Allocator output signals

### C.12 Functional Behaviour

### C.13 Allocation Flow

1. Upstream presents AXID[5:0] and asserts REQ_VALID.
2. Stall Logic reads occupancy[AXID] from the Occupancy Counter Array combinationally.
3. If occupancy[AXID] < 4: STALL is deasserted, alloc_en is asserted.
4. Allocate Logic reads seqnum[1:0] from the Seqnum Counter Array for the requesting AXID.
5. TXN_TAG = {AXID, seqnum} is driven; TAG_VALID is asserted.
6. Occupancy Counter Array increments for AXID.
7. Seqnum Counter Array increments (mod 4) for AXID.

### C.14 Stall Flow

1. If occupancy[AXID] == 4: STALL is asserted to upstream.
2. Upstream holds REQ_VALID and AXID stable.

3. Stall is released as soon as Free Logic decrements occupancy for that AXID (i.e., on the next `WR_FREE_VALID` or `RD_FREE_VALID` for a tag belonging to that AXID).

### C.15 Free Flow

1. Response Tracker asserts `WR_FREE_VALID` with `WR_FREE_TAG` when a BRESP is emitted (write path).
2. Read Txn Table asserts `RD_FREE_VALID` with `RD_FREE_TAG` when `rob_last` fires (read path).
3. Free Logic extracts `AXID = tag[7:2]` and asserts `dec_occ` to the Occupancy Counter Array.
4. This may immediately release a stall on that AXID if Stall Logic observes the updated occupancy.

### C.16 Timing Notes

- Stall Logic is purely combinational: `STALL` is available in the same cycle that `AXID` is presented.
- Allocate Logic is registered: `TXN_TAG` and `TAG_VALID` are valid on the cycle *following* `alloc_en`.
- Counter updates (inc/dec) are registered and take effect on the following clock edge. A free in cycle $N$ releases the stall in cycle $N{+}1$.

### C.17 Edge Cases

- **Simultaneous alloc and free on same AXID:** If `alloc_en` and `dec_occ` fire in the same cycle for the same AXID, occupancy is held constant (net zero change). This is a valid steady-state operating condition.
- **Error path:** Error Handler allocates a tag identically to valid transactions. The resulting tag is embedded in a NOP packet (`pkt_type = Err_write / Err_read`). The slot is freed normally on MC Core error completion.
- **Underflow protection:** The Occupancy Counter Array must not decrement below zero. This is guaranteed by construction — a free is only issued when a tag is in-flight — but a design-time assertion should be included in the verification environment.

## D Response Tracker

### D.1 Overview

The Response Tracker is a write-path block responsible for aggregating per-packet MC Core completions into a single AXI B channel response per transaction. It tracks every outstanding write transaction from tag allocation to final BRESP emission.

Its responsibilities are:

- Store per-transaction metadata at allocation time: expected packet count, completed packet count, error status.
- Increment the completed packet counter on each `WR_ACK` completion received from MC Core.
- Assert `txn_done` when all expected packets for a transaction have completed.

- Enforce AXI4 B channel ordering: BRESP is emitted in-order per AXID using a per-AXID head pointer.
- Propagate error status: if any packet in a burst returns an error completion, the sticky error bit is set and the transaction generates `BRESP = DECERR`.
- Free the occupied Tag Allocator slot on BRESP emission via the `WR_FREE` port.

**Note on abstraction levels.** The full CIF pipeline diagram shows MC Core as the sole input to the Response Tracker. At the internal logic level, the Tag Allocator also provides an allocation signal — this is a resource management path (writing `exp_pkts` at tag assignment time) that is separate from the data completion path. Both are correct; they operate at different abstraction levels.

## D.2 Design Parameters

| Parameter | Value | Description |
|---|---|---|
| `AXID_W` | 6 | AXI ID width; 64 unique IDs |
| `SEQNUM_W` | 2 | Seqnum width |
| `TXN_TAG_W` | 8 | Tag width = AXID_W + SEQNUM_W |
| `RT_DEPTH` | 256 | Register file entries = $2^8$ |
| `EXP_PKTS_W` | 8 | Expected packets width (covers AWLEN+1, max 256) |
| `CMP_PKTS_W` | 8 | Completed packets counter width |
| `N_AXIDS` | 64 | Head pointer array depth |
| `HEAD_W` | 8 | Head pointer width = TXN_TAG_W |

Table 39: Response Tracker parameters

## D.3 Internal Blocks

## D.4 Register File

- **Structure:** 256-entry register file. Each entry stores:
  - `exp_pkts[7:0]` — expected packet count (= `AWLEN+1`)
  - `cmp_pkts[7:0]` — completed packet count
  - `status[3:0]` — MC Core status code from most recent error completion
  - `err_sticky[0]` — sticky error bit; set on first error completion, never cleared until entry is freed
- **Addressing:** `addr[7:0] = TXN_TAG[7:0] = {AXID[5:0], seqnum[1:0]}`. Tag directly indexes the register file — no translation required.
- **Write ports:**
  - Alloc Write Logic: writes `exp_pkts`, initialises `cmp_pkts = 0`, `err_sticky = 0` on allocation.
  - Completion Update Logic: increments `cmp_pkts`, optionally sets `err_sticky` and `status` on each `WR_ACK` completion.

  - Free Logic: clears the entry on BRESP emission.

- **Read ports:** Continuously exposes `cmp_pkts` and `exp_pkts` to Compare Logic; exposes `err_sticky` and `status` to Emit Logic.

### D.5 Alloc Write Logic

- Activated on `TAG_VALID` from the Tag Allocator.
- Receives `TXN_TAG[7:0]` and `AWLEN[7:0]`.
- Computes `exp_pkts = AWLEN + 1`.
- Issues `write_en` to the Register File at address `TXN_TAG[7:0]`, writing `exp_pkts` and initialising `cmp_pkts = 0`, `err_sticky = 0`.

### D.6 Completion Update Logic

- Taps the MC Core completion bus: `RESP_VALID`, `resp_type[1:0]`, `tag[7:0]`, `status[3:0]`.
- **Filter:** acts only when `resp_type == 2'b01` (WR_ACK). RD_DATA completions (`resp_type == 2'b00`) are destined for the ROB and RTT and are ignored here.
- On a normal WR_ACK (`resp_type == 2'b01`, no error): asserts `inc_en` to the Register File at address `tag[7:0]`, incrementing `cmp_pkts` by 1.
- On an error completion (`resp_type == 2'b10`): increments `cmp_pkts` and additionally sets `err_sticky = 1` and latches `status[3:0]`. The sticky bit is never cleared by subsequent completions — one error poisons the entire transaction.

### D.7 Compare Logic

- Purely combinational.
- Reads `cmp_pkts[7:0]` and `exp_pkts[7:0]` from the Register File for the entry currently being updated.
- Asserts `txn_done` as a one-cycle pulse when `cmp_pkts == exp_pkts`.
- `txn_done` is forwarded to Emit Logic to trigger BRESP evaluation.

### D.8 Head Pointer Array

- **Structure:** 64 × 8-bit registers, one per AXID. Each register holds the `TXN_TAG` of the next transaction expected to emit BRESP for that AXID.
- **Init:** Written on first allocation for an AXID (occupancy 0→1). On subsequent allocations for the same AXID, the head pointer is not updated — it is owned by Emit Logic from that point forward.
- **Advance:** Emit Logic increments the head pointer (mod 4 on the seqnum field) after each BRESP is emitted for that AXID.
- **Purpose:** Enforces AXI4 B channel ordering — BRESP is only emitted for the transaction at the head, not for any later-completing transaction of the same AXID.

### D.9 Emit Logic

- Checks whether the transaction at `head_tag[AXID]` is done: reads `txn_done` from Compare Logic and confirms the completing tag matches the current head pointer for that AXID.
- If head slot is done:
  - Reads `err_sticky` and `status` from the Register File.
  - Asserts `BRESP_VALID` to the B Response Generator.
  - Drives `BID[5:0] = head_tag[7:2]` (AXID recovered from tag).
  - Drives `BRESP[1:0] = OKAY` if `err_sticky == 0`; `DECERR` if `err_sticky == 1`.
  - Asserts `advance_head` to the Head Pointer Array for the relevant AXID.
  - Asserts `emit_done` with `tag[7:0]` to Free Logic.
- If head slot is not yet done: Emit Logic holds. Later-completing transactions for the same AXID are buffered in the Register File until the head slot completes.

### D.10 Free Logic

- Activated by `emit_done` from Emit Logic.
- Asserts `clear_en` to the Register File at address `tag[7:0]`, clearing the entry in the same cycle as BRESP emission.
- Simultaneously asserts `WR_FREE_VALID` and drives `WR_FREE_TAG[7:0]` to the Tag Allocator's WR_FREE port, decrementing the occupancy counter for the freed AXID.

### D.11 Interface Signals

### D.12 Inputs

| Signal | Width | Source | Description |
|---|---|---|---|
| `TXN_TAG[7:0]` | 8 | Tag Allocator | Allocated tag at write transaction time |
| `AWLEN[7:0]` | 8 | Tag Allocator | Burst length; `exp_pkts = AWLEN+1` |
| `TAG_VALID` | 1 | Tag Allocator | Allocation strobe |
| `RESP_VALID` | 1 | MC Core | Completion valid |
| `resp_type[1:0]` | 2 | MC Core | 00=RD_DATA (ignored), 01=WR_ACK, 10=ERR |
| `tag[7:0]` | 8 | MC Core | Completion tag; used as register file address |
| `status[3:0]` | 4 | MC Core | Error status; latched on error completion |

Table 40: Response Tracker input signals

### D.13 Outputs

| Signal | Width | Destination | Description |
|---|---|---|---|

| | | | |
|---|---|---|---|
| BRESP_VALID | 1 | B Response Generator | BRESP ready to emit |
| BID[5:0] | 6 | B Response Generator | AXI ID recovered from head_tag[7:2] |
| BRESP[1:0] | 2 | B Response Generator | OKAY or DECERR |
| WR_FREE_VALID | 1 | Tag Allocator (WR_FREE port) | Free strobe; coincident with BRESP emit |
| WR_FREE_TAG[7:0] | 8 | Tag Allocator (WR_FREE port) | Tag being freed |

Table 41: Response Tracker output signals

### D.14 Internal Signals

| Signal | Width | From | To | Description |
|---|---|---|---|---|
| write_en | 1 | Alloc Write Logic | Register File | Write strobe on allocation |
| wr_addr[7:0] | 8 | Alloc Write Logic | Register File | = TXN_TAG[7:0] |
| exp_pkts[7:0] | 8 | Alloc Write Logic | Register File | = AWLEN+1 |
| inc_en | 1 | Completion Update Logic | Register File | Increment cmp_pkts strobe |
| err_set | 1 | Completion Update Logic | Register File | Set err_sticky on error |
| inc_addr[7:0] | 8 | Completion Update Logic | Register File | = tag[7:0] from MC Core |
| cmp_pkts[7:0] | 8 | Register File | Compare Logic | Current completed count |
| exp_pkts[7:0] | 8 | Register File | Compare Logic | Expected count (read port) |
| err_sticky | 1 | Register File | Emit Logic | Sticky error flag |
| txn_done | 1 | Compare Logic | Emit Logic | Pulse: all packets completed |
| head_tag[7:0] | 8 | Head Pointer Array | Emit Logic | Next tag to emit per AXID |
| advance_head | 1 | Emit Logic | Head Pointer Array | Increment head pointer for AXID |
| emit_done | 1 | Emit Logic | Free Logic | BRESP emitted; trigger free |
| clear_en | 1 | Free Logic | Register File | Clear entry strobe |
| clear_addr[7:0] | 8 | Free Logic | Register File | = tag of emitted transaction |

Table 42: Response Tracker internal signals

### D.15 Functional Behaviour

### D.16 Allocation Flow

1. Tag Allocator asserts TAG_VALID with TXN_TAG[7:0] and AWLEN[7:0].
2. Alloc Write Logic computes exp_pkts = AWLEN + 1.

3. Register File is written at address `TXN_TAG[7:0]`: `exp_pkts` stored, `cmp_pkts = 0`, `err_sticky = 0`.
4. If this is the first in-flight transaction for this AXID (occupancy 0→1), Head Pointer Array is initialised with `TXN_TAG[7:0]`.

### D.17 Completion Flow

1. MC Core asserts `RESP_VALID` with `resp_type == 2'b01` (WR_ACK) and `tag[7:0]`.
2. Completion Update Logic asserts `inc_en` at address `tag[7:0]`: `cmp_pkts` incremented by 1.
3. If `resp_type == 2'b10` (ERR): additionally sets `err_sticky = 1` and latches `status[3:0]`.
4. Compare Logic evaluates `cmp_pkts == exp_pkts` combinationally; asserts `txn_done` if true.

### D.18 Emit Flow

1. `txn_done` pulses for a completing transaction.
2. Emit Logic checks whether the completing tag matches `head_tag` for that AXID.
3. If match (in-order): reads `err_sticky` from Register File, drives `BRESP_VALID`, `BID`, and `BRESP` (OKAY or DECERR) to B Response Generator. Asserts `advance_head` and `emit_done`.
4. If no match (out-of-order): transaction is complete in the Register File but the head has not yet completed. Emit Logic holds. BRESP is deferred until the head slot completes.

### D.19 Free Flow

1. `emit_done` triggers Free Logic in the same cycle as BRESP emission.
2. Free Logic asserts `clear_en` to Register File: entry is invalidated.
3. `WR_FREE_VALID` and `WR_FREE_TAG` are asserted to Tag Allocator, decrementing occupancy for the freed AXID.

### D.20 Timing Notes

- Alloc Write Logic is registered: Register File write takes effect the cycle after `TAG_VALID`.
- Completion Update Logic is registered: `cmp_pkts` increment takes effect the cycle after `RESP_VALID`.
- Compare Logic is combinational: `txn_done` appears in the same cycle the increment is visible — one cycle after the final `RESP_VALID`.
- Emit Logic is registered: `BRESP_VALID` is asserted the cycle after `txn_done` is evaluated.
- Free Logic fires in the same cycle as BRESP emission: `clear_en` and `WR_FREE_VALID` are asserted without additional latency.
- Simultaneous allocation and completion on the same tag are not possible by construction — a tag is only re-allocated after its slot is freed, which occurs after BRESP emission.

### D.21 Edge Cases

- **Single-packet transaction (AWLEN = 0):** `exp_pkts = 1`. `txn_done` fires on the first and only WR_ACK. Handled identically to multi-packet transactions.
- **Partial error in a burst:** If one packet in a burst returns ERR and others return WR_ACK, `err_sticky` is set on the first error. All subsequent completions still increment `cmp_pkts` normally. On BRESP emission, `BRESP = DECERR` regardless of how many packets succeeded.
- **Out-of-order completion within an AXID:** AXI4 requires B channel responses to be in-order per AXID. If a later transaction of an AXID completes before the head transaction, Emit Logic holds the BRESP until the head slot is done. The later transaction's completion is safely held in the Register File.
- **RD_DATA completions:** Ignored entirely by Completion Update Logic. These are processed by the ROB and RTT on the read path.
- **Over-completion protection:** `cmp_pkts` must never exceed `exp_pkts`. Guaranteed by construction since MC Core returns exactly one WR_ACK per issued write packet. A design-time assertion should be included in the verification environment.
- **Simultaneous alloc and completion on same AXID (different seqnums):** These target different register file entries (different `tag[7:0]` addresses) — no conflict.

# E Read Transaction Table (RTT)

## E.1 Overview

The Read Transaction Table (RTT) is a per-transaction tracking structure for the read path. It is allocated at tag assignment time and tracks completion progress for every outstanding read transaction. When all expected packets for a transaction have been received from MC Core, the RTT asserts `rob_last`, signalling the final beat to the ROB and Merge Logic. The entry is freed in the same cycle.

The RTT operates independently of the ROB — both blocks tap the same MC Core completion bus, but serve different purposes: the ROB stores and reorders data; the RTT tracks transaction-level progress and drives `rob_last`.

## E.2 Design Parameters

| Parameter | Value | Description |
|---|---|---|
| AXID_W | 6 | AXI ID width |
| SEQNUM_W | 2 | Seqnum width |
| TXN_TAG_W | 8 | Tag width = AXID_W + SEQNUM_W |
| RTT_DEPTH | 256 | Entries = $2^{\text{AXID\_W+SEQNUM\_W}}$ |
| EXP_PKTS_W | 8 | Expected packets width (covers ARLEN+1, max 256) |
| CMP_PKTS_W | 8 | Completed packets counter width |

Table 43: RTT parameters

### E.3 Internal Blocks

### E.4 RTT Register File

- **Structure:** 256-entry register file. Each entry stores:
  - `AXID[5:0]` — AXI ID of the transaction
  - `exp_pkts[7:0]` — expected packet count (= `ARLEN+1`)
  - `cmp_pkts[7:0]` — completed packet count (incremented on each MC Core completion)
- **Addressing:** `addr[7:0] = TXN_TAG[7:0] = {AXID[5:0], seqnum[1:0]}`. The tag directly indexes the register file — no translation required.
- **Write ports:**
  - Alloc Write Logic writes `exp_pkts` on allocation.
  - Completion Update Logic increments `cmp_pkts` on each `RD_DATA` completion.
  - Free Logic clears the entry (zeroes `cmp_pkts`, invalidates) on `rob_last`.
- **Read port:** Continuously exposes `cmp_pkts` and `exp_pkts` to Compare Logic.

### E.5 Alloc Write Logic

- Activated on `TAG_VALID` from the Tag Allocator.
- Receives `TXN_TAG[7:0]` and `ARLEN[7:0]`.
- Computes `exp_pkts = ARLEN + 1`.
- Issues `write_en` to the RTT Register File at address `TXN_TAG[7:0]`, writing `exp_pkts` and initialising `cmp_pkts = 0`.

### E.6 Completion Update Logic

- Taps the MC Core completion bus: `RESP_VALID`, `resp_type[1:0]`, `tag[7:0]`.
- **Filter:** acts only when `resp_type == 2'b00` (RD_DATA). WR_ACK completions (`resp_type == 2'b01`) are destined for the Response Tracker and are ignored. ERR completions for read transactions (`resp_type == 2'b10`) are treated as a valid completion event — `cmp_pkts` is still incremented so the transaction reaches `rob_last` and completes normally; error status propagation is handled by the ROB and R Response Generator.
- Asserts `inc_en` to the RTT Register File at address `tag[7:0]`, incrementing `cmp_pkts` by 1.

### E.7 Compare Logic

- Purely combinational.
- Reads `cmp_pkts[7:0]` and `exp_pkts[7:0]` from the RTT Register File for the entry currently being updated.
- Asserts `rob_last` as a one-cycle pulse when `cmp_pkts == exp_pkts`.

- `rob_last` is forwarded to two destinations simultaneously: Free Logic (to clear the entry) and the ROB (passed through to Merge Logic coincident with the final data beat).

### E.8 Free Logic

- Activated by `rob_last` from Compare Logic.
- Asserts `clear_en` to the RTT Register File at address `tag[7:0]`, clearing the entry in the same cycle as `rob_last`.
- Simultaneously asserts `RD_FREE_VALID` and drives `RD_FREE_TAG[7:0]` to the Tag Allocator's RD_FREE port, triggering occupancy decrement for the freed AXID.

### E.9 Interface Signals

### E.10 Inputs

| Signal | Width | Source | Description |
|---|---|---|---|
| `TXN_TAG[7:0]` | 8 | Tag Allocator | Allocated tag; indexes the register file |
| `ARLEN[7:0]` | 8 | Tag Allocator | Burst length; `exp_pkts = ARLEN+1` |
| `TAG_VALID` | 1 | Tag Allocator | Allocation strobe |
| `RESP_VALID` | 1 | MC Core | Completion valid |
| `resp_type[1:0]` | 2 | MC Core | 00=RD_DATA, 01=WR_ACK, 10=ERR; RTT acts on RD_DATA and ERR |
| `tag[7:0]` | 8 | MC Core | Completion tag; used as register file address |

Table 44: RTT input signals

### E.11 Outputs

| Signal | Width | Destination | Description |
|---|---|---|---|
| `rob_last` | 1 | ROB / Merge Logic | Pulse on final packet of a transaction |
| `RD_FREE_VALID` | 1 | Tag Allocator (RD_FREE port) | Free strobe; coincident with `rob_last` |
| `RD_FREE_TAG[7:0]` | 8 | Tag Allocator (RD_FREE port) | Tag being freed |

Table 45: RTT output signals

### E.12 Internal Signal Table

| Signal | Width | From | To | Description |
|---|---|---|---|---|
| `write_en` | 1 | Alloc Write Logic | RTT Reg File | Write strobe on allocation |

| | | | | |
|---|---|---|---|---|
| wr_addr[7:0] | 8 | Alloc Write Logic | RTT Reg File | = TXN_TAG[7:0] |
| exp_pkts[7:0] | 8 | Alloc Write Logic | RTT Reg File | = ARLEN+1 |
| inc_en | 1 | Completion Update Logic | RTT Reg File | Increment cmp_pkts strobe |
| inc_addr[7:0] | 8 | Completion Update Logic | RTT Reg File | = tag[7:0] from MC Core |
| cmp_pkts[7:0] | 8 | RTT Reg File | Compare Logic | Current completed count |
| exp_pkts[7:0] | 8 | RTT Reg File | Compare Logic | Expected count (read port) |
| rob_last | 1 | Compare Logic | Free Logic, ROB | One-cycle pulse on final completion |
| clear_en | 1 | Free Logic | RTT Reg File | Clear entry strobe |
| clear_addr[7:0] | 8 | Free Logic | RTT Reg File | = tag[7:0] of completing txn |

Table 46: RTT internal signals

### E.13 Functional Behaviour

### E.14 Allocation Flow

1. Tag Allocator asserts TAG_VALID with TXN_TAG[7:0] and ARLEN[7:0].
2. Alloc Write Logic computes exp_pkts = ARLEN + 1.
3. RTT Register File is written at address TXN_TAG[7:0]: exp_pkts is stored, cmp_pkts is initialised to zero.

### E.15 Completion Flow

1. MC Core asserts RESP_VALID with resp_type == 2'b00 (RD_DATA) and tag[7:0].
2. Completion Update Logic asserts inc_en at address tag[7:0]: cmp_pkts is incremented by 1.
3. Compare Logic reads the updated cmp_pkts and exp_pkts combinationally.
4. If cmp_pkts == exp_pkts: rob_last is asserted as a one-cycle pulse.

### E.16 Free Flow

1. rob_last pulse triggers Free Logic in the same cycle.
2. Free Logic asserts clear_en to the RTT Register File: the entry is invalidated.
3. Free Logic simultaneously asserts RD_FREE_VALID with RD_FREE_TAG to the Tag Allocator, decrementing the occupancy counter for the freed AXID.

### E.17 rob_last Routing

rob_last is a sideband signal to Merge Logic. It travels in parallel with, not through, the ROB. The ROB emits data[511:0], status[3:0], and AXID[5:0] to Merge Logic; the RTT emits rob_last to Merge Logic coincident with the final data beat. Merge Logic combines both to assert RLAST on the AXI R channel. There is no feedback path from the RTT into the ROB's internal logic.

### E.18 Timing Notes

- Alloc Write Logic is registered: the register file write takes effect on the cycle following `TAG_VALID`.
- Completion Update Logic is registered: `cmp_pkts` increment takes effect on the cycle following `RESP_VALID`.
- Compare Logic is combinational: `rob_last` is available in the same cycle as the `cmp_pkts` update is visible. In practice this means `rob_last` appears one cycle after the final `RESP_VALID`.
- Free Logic fires in the same cycle as `rob_last`: the register file entry is cleared and `RD_FREE_VALID` is asserted without additional latency.
- Simultaneous allocation and completion on the same tag are not possible by construction — a tag is only allocated after its previous occupancy slot is freed, and a completion can only arrive for an in-flight tag.

### E.19 Edge Cases

- **Single-beat transaction (ARLEN = 0):** `exp_pkts = 1`. `rob_last` is asserted on the first and only completion. Handled identically to multi-beat transactions.
- **Error completion:** `resp_type == 2'b10` (ERR) for a read transaction is treated as a valid completion event. `cmp_pkts` is incremented normally so the transaction progresses to `rob_last`. Error status is stored in the ROB slot and forwarded to the R Response Generator as DECERR. The RTT does not need to distinguish error from normal completions.
- **WR_ACK completions:** `resp_type == 2'b01` completions are ignored by the RTT entirely. They are processed by the Response Tracker on the write path.
- **Over-completion protection:** `cmp_pkts` must never exceed `exp_pkts`. This is guaranteed by construction since MC Core returns exactly one completion per issued read packet. A design-time assertion should be included in the verification environment.

## F Reorder Buffer (ROB)

### F.1 Overview

The Reorder Buffer (ROB) absorbs out-of-order read completions from MC Core and emits them in-order per AXID to Merge Logic. Depth is tied to `MC_RD_BUF = 64`. Since `RESP_READY` must always be held high, the ROB can never stall incoming completions — backpressure is instead applied by halting new read requests when occupancy reaches the watermark threshold.

#### Input Signals

| Signal | Width | Source | Description |
|---|---|---|---|
| `alloc_req` | 1 | Tag Allocator | Pulse: allocate a new ROB slot |
| `AXID[5:0]` | 6 | Tag Allocator | AXI ID of transaction being allocated |
| `seqnum[1:0]` | 2 | Tag Allocator | Seqnum of transaction being allocated |
| `tag[7:0]` | 8 | Tag Allocator | Full tag (used to write Lookup Table) |

| | | | |
|---|---|---|---|
| RESP_VALID | 1 | MC Core | Completion valid strobe |
| resp_type[1:0] | 2 | MC Core | 00=RD_DATA, 01=WR_ACK, 10=ERR; ROB acts only on RD_DATA |
| tag[7:0] | 8 | MC Core | Completion tag; [7:2]=AXID, [1:0]=seqnum |
| data[511:0] | 512 | MC Core | Read data payload |
| status[3:0] | 4 | MC Core | Completion status/error code |

Table 47: ROB input signals

### Output Signals

| Signal | Width | Destination | Description |
|---|---|---|---|
| data[511:0] | 512 | Merge Logic | In-order read data beat |
| status[3:0] | 4 | Merge Logic | Status for emitted beat |
| AXID[5:0] | 6 | Merge Logic | AXID of emitted beat |
| rob_last | 1 | Merge Logic | Asserted on final beat of a transaction; sourced from RTT, passed through |
| PKT_READY | 1 | Packet FIFO | Deasserted when watermark hit; halts new read requests |
| RESP_READY | 1 | MC Core | Tied high permanently |

Table 48: ROB output signals

## F.2 Blocks

## F.3 Free List

- 64-bit bitmask. `1 = free`, `0 = occupied`.
- Allocate: priority encoder selects lowest free bit, clears it, outputs `free_slot_idx[5:0]`.
- Free: Emit Logic sets the bit at `slot_idx` via `free_req`.
- Outputs `free_slot_idx` to Lookup Table (write), ROB BRAM (init), and Head Pointer Array (first allocation).

## F.4 Occupancy Counter

- Up/down counter, range 0–64.
- Incremented by `alloc_req` from Free List.
- Decremented by `free_req` from Emit Logic.
- Asserts `watermark_hit` when count $\geq$ `MC_RD_BUF` $-$ `ROB_WATERMARK` $= 38$.

## F.5 Lookup Table

- 256-entry register file, addressed by {`AXID[5:0], seqnum[1:0]`} (8-bit address).
- Each entry stores `slot_idx[5:0]`.
- Write port: Tag Allocator writes `slot_idx` at {`AXID, seqnum`} on allocation.
- Read port: Completion Input reads `slot_idx` using {`tag[7:2], tag[1:0]`}.
- Clear: Emit Logic clears entry at {`AXID, seqnum`} after emit.

### F.6 ROB BRAM

- Dual-port BRAM, 64 × 517 bits per slot: `valid[0], data[511:0], status[3:0]`.
- Write port: Completion Input writes `data`, `status`, `valid=1` at `slot_idx`.
- Write port (init): Free List writes `valid=0` at `free_slot_idx` on allocation.
- Read port: Head Pointer Array supplies `head[AXID]` as read address; all 64 head slots checked in parallel each cycle.
- 1-cycle read latency — Emit Logic must account for this in its valid check timing.

### F.7 Completion Input

- Receives `RESP_VALID, resp_type[1:0], tag[7:0], data[511:0], status[3:0]` from MC Core.
- **Filters on `resp_type`:** acts only when `resp_type == 2'b00` (RD_DATA). Completions with `resp_type == 2'b01` (WR_ACK) are destined for the Response Tracker and are ignored by the ROB. ERR completions (`resp_type == 2'b10`) for read transactions are written into the ROB with the error status preserved so DECERR can be forwarded to the R Response Generator.
- Extracts `AXID = tag[7:2]`, `seqnum = tag[1:0]`.
- Reads `slot_idx` from Lookup Table using {`AXID, seqnum`}.
- Writes `data`, `status`, `valid=1` to ROB BRAM at `slot_idx`.

### F.8 Head Pointer Array

- 64 registers of 6 bits each, one per AXID. Stores slot index of next expected beat.
- Written on first allocation only: `head[AXID] = free_slot_idx` when the AXID has no in-flight slots (occupancy transitions 0→1 for that AXID). On subsequent allocations for the same AXID, `head[AXID]` is *not* updated — it is owned exclusively by Emit Logic from that point forward.
- Advanced by Emit Logic after each emit: incremented to the next expected slot index.
- Reset to the next `free_slot_idx` only when all in-flight slots for an AXID have been emitted and a new allocation arrives (occupancy returns to 0 then rises again).
- All 64 head pointers drive the ROB BRAM read port simultaneously each cycle.

### F.9 Emit Logic

- Reads `valid` bits from all 64 head slots out of ROB BRAM each cycle.
- Priority encoder selects lowest AXID whose head slot has `valid=1`.
- On selection: reads `data` and `status` from BRAM; emits to Merge Logic along with `AXID` and `rob_last`. `rob_last` is *not* generated inside the ROB — it is sourced from the Read Transaction Table (RTT), which asserts it when `completed_packets == expected_packets` for that transaction. The ROB passes it through to Merge Logic coincident with the final data beat.
- After emit:
  - Clears `valid` in BRAM slot.
  - Advances `head[AXID]` in Head Pointer Array.
  - Sends `free_req + slot_idx` to Free List (sets bitmask bit).
  - Decrements Occupancy Counter via `free_req`.
  - Clears Lookup Table entry at {`AXID, seqnum`}.

- Maximum one emission per cycle (single R channel path).

### F.10 PKT_READY Control

- Purely combinational.
- Asserts `PKT_READY=0` to Packet FIFO when `watermark_hit` is high.
- Halts new read requests to MC Core; prevents ROB overflow since `RESP_READY` cannot be deasserted.

### F.11 Allocation Path

1. Tag Allocator sends `alloc_req` (pulse) with `AXID`, `seqnum`, `tag`.
2. Free List pops `free_slot_idx` (priority encoder on bitmask), clears the bit.
3. Free List writes `slot_idx` to Lookup Table at {`AXID, seqnum`}.
4. Free List initializes ROB BRAM slot: `valid=0` at `free_slot_idx`.
5. Free List stores `free_slot_idx` into `head[AXID]` (first allocation for that AXID only).
6. Occupancy Counter increments.

### F.12 Completion Path

1. MC Core asserts `RESP_VALID` with `tag[7:0]`, `data[511:0]`, `status[3:0]`.
2. Completion Input extracts `AXID = tag[7:2]`, `seqnum = tag[1:0]`.
3. Completion Input reads `slot_idx` from Lookup Table.
4. Completion Input writes `data`, `status`, `valid=1` to ROB BRAM at `slot_idx`.
5. `RESP_READY` is tied high — MC Core never stalled on this path.

### F.13 Emit Path

1. Each cycle, all 64 `head[AXID]` values drive the ROB BRAM read port.
2. BRAM returns `valid` bits for all 64 head slots (1-cycle latency).
3. Priority encoder in Emit Logic selects lowest AXID with `valid=1`.
4. Emit Logic reads `data` and `status` from selected slot; forwards to Merge Logic with `AXID` and `last_beat`.
5. Slot is freed: `valid` cleared, `head[AXID]` advanced, Free List bit set, Occupancy Counter decremented, Lookup Table entry cleared.

### F.14 Backpressure & Overflow Prevention

| Signal | Description |
|---|---|
| `watermark_hit` | Asserted when occupancy $\geq 38$. Drives PKT_READY control. |
| `PKT_READY=0` | Stops Packet FIFO from issuing new read requests to MC Core. |
| `RESP_READY=1` | Tied high permanently. ROB can never stall MC Core completions. |

Watermark headroom = `ROB_WATERMARK = 26`. Ensures all requests already accepted by MC Core can return completions without overflowing the ROB.

### F.15 Key Parameters

| Parameter | Value |
|---|---|
| ROB depth | 64 entries (`MC_RD_BUF`) |
| Slot width | 517 bits (1 valid + 512 data + 4 status) |
| Free list width | 64 bits |
| Lookup table | 256 entries × 6-bit slot index |
| Head pointers | 64 × 6-bit registers |
| Watermark | 38 (64 − 26) |
| Max emissions | 1 per cycle |

### F.16 Timing Notes

- ROB BRAM has 1-cycle read latency. Emit Logic valid check must be pipelined accordingly — register the read address, use valid bit one cycle after presenting `head[AXID]`.
- Priority encoder across 64 valid bits is the critical path in Emit Logic. May need pipelining at high frequencies.
- Allocation and completion can occur in the same cycle — write port (completion) and init write (allocation) are independent BRAM ports.
- `watermark_hit` is combinational from the Occupancy Counter — no additional latency before `PKT_READY` deasserts.
- Simultaneous `alloc_req` and `free_req` on the Free List bitmask in the same cycle is safe — they always target different bits (alloc clears the lowest free bit; free sets the bit of the just-emitted slot, which is by definition occupied). No read-modify-write conflict arises.